

Análise Técnico-Científica de uma Implementação de Ray Tracing

Technical-Scientific Analysis of a Ray Tracing Implementation

Yang Ricardo Barcellos Miranda [ymiranda@inf.puc-rio.br]

Resumo. Este artigo apresenta uma análise técnico-científica da implementação de ray tracing desenvolvida no Projeto 1 da disciplina de INF2608. O objetivo é verificar, a partir do código efetivamente executável, quais componentes geométricos, fotométricos e amostrais foram materializados, como eles se relacionam com os slides da disciplina e em que medida o resultado se aproxima de referências selecionadas de PBRT 4e. O texto enfatiza as explicações matemáticas sobre formação da imagem, interseções, iluminação local, amostragem, reflexão, refração e geometria triangular, e consolida a rastreabilidade detalhada em quadros finais de referência e evidência com notas sobre páginas dos slides e linhas de código.

Abstract. This paper presents a technical-scientific analysis of the ray tracing implementation developed for the INF2608 course project. Its goal is to verify, from the executable code itself, which geometric, photometric, and sampling components were materialized, how they relate to the course slides, and to what extent the result aligns with selected PBRT 4e references. The text emphasizes the mathematical discussion of image formation, intersections, local illumination, sampling, reflection, refraction, and triangle geometry, and consolidates detailed traceability in final reference-and-evidence tables with notes pointing to slide pages and source-code lines.

Palavras-chave: traçado de raios, computação gráfica, PBRT, amostragem, BVH, renderização

Keywords: ray tracing, computer graphics, PBRT, sampling, BVH, rendering

Entrega: 10/05/2026

1 Introdução

Este artigo analisa a implementação de ray tracing disponível no repositório do Projeto 1 [Miranda, 2026]. O objeto principal da análise é o código em `src/ray_tracing_2/`, confrontado com os tópicos dos slides da disciplina, de autoria de Waldemar Celes [Celes, 2026b,c,a], e com referências selecionadas de *Physically Based Rendering* (PBRT 4e) [Pharr *et al.*, 2023]. O objetivo não é recontar genericamente o que um ray tracer deveria fazer, mas examinar o que esta base entrega de forma observável, quais decisões algorítmicas sustentam cada efeito visual e onde o projeto simplifica o conteúdo teórico.

A organização do texto privilegia a leitura do pipeline do traçador, do núcleo geométrico e fotométrico, das extensões ópticas e amostrais, da geometria triangular e das estruturas de aceleração. As imagens principais ficam reunidas em uma seção específica de evidências visuais, na qual cada figura é acompanhada por uma leitura do que ela comprova, pelos metadados mais relevantes do render e pelo comando gerador correspondente.

Para preservar a fluidez do corpo principal, a rastreabilidade detalhada aparece nos anexos finais. O Anexo A reúne quadros consolidados de *conceitos de referência*, *evidência principal* e *nota*, em que as notas sintetizam

as páginas dos slides, as seções mais próximas em PBRT e as linhas de código mais relevantes. O Anexo B mantém duas vistas complementares do diagrama de classes: uma versão inicial, mais sintética, e uma versão estendida que incorpora a infraestrutura experimental hoje presente na base.

2 Núcleo Geométrico e Fotométrico do Traçador

2.1 Pipeline geral do traçador

O pipeline do traçador pode ser lido como uma composição de quatro etapas. Primeiro, o filme define a amostragem do pixel; depois, a câmera converte a amostra em um raio primário; em seguida, a cena resolve a primeira interseção visível; por fim, o material associado ao *hit*¹ calcula a contribuição local e, quando necessário, dispara raios secundários. Em forma resumida, a cor do pixel pode ser entendida como

$$C_{ij} = \frac{1}{N} \sum_{k=1}^N L(r_{ij,k}),$$

em que cada raio $r_{ij,k}$ nasce na câmera, atravessa

¹Neste texto, *hit* designa o registro da interseção válida selecionada pelo teste de visibilidade; *closest hit* é a interseção com menor parâmetro positivo t ao longo do raio.

a cena e devolve uma estimativa da radiância naquela direção. O caso básico do projeto usa $N = 1$ com uma amostra central; os casos com antialiasing substituem essa leitura pontual por uma média sobre o domínio do pixel.

Do ponto de vista arquitetural, esse encaideamento tem uma consequência importante: o núcleo do renderizador continua pequeno mesmo quando a ótica fica mais rica. `Film.render()` gera amostras, `Camera.generate_ray()` produz raios, `Scene.compute_intersection()` escolhe o *closest hit* e `Scene.trace_ray()` delega ao material a decisão de permanecer no sombreado local ou seguir por recursão. Essa separação é central para entender por que reflexão, refração e transmitância puderam ser adicionadas sem substituir o pipeline original.

2.2 Raio paramétrico, visibilidade e registro de interseção

O ponto de partida do Slide 4 é a modelagem do raio por

$$\mathbf{r}(t) = \mathbf{o} + t \hat{\mathbf{d}}, \quad t \geq 0,$$

em que $\mathbf{o}$ é a origem, $\hat{\mathbf{d}}$ é a direção normalizada e t parametriza a posição ao longo do feixe [Celes, 2026b, pp. 7–9]. Em óptica geométrica, essa escrita abstrai a propagação retilínea da luz em meios homogêneos; em computação gráfica, ela transforma visibilidade em busca do menor $t > 0$ compatível com uma superfície.

O repositório segue exatamente essa lógica. O raio é carregado por `Ray`; o registro da melhor interseção é encapsulado em `Hit`; e a cena trabalha com um único acumulador que preserva a interseção frontal mais próxima. A distinção entre `front_face` e `backfacing`, calculada em `Hit.set_face_normal()`, começa como um detalhe geométrico de orientação de normal, mas depois se torna decisiva para os materiais refrativos, pois separa entrada e saída de meios diferentes.

2.3 Interseções básicas e controle numérico de ε

Nos slides, o plano é descrito pela equação implícita

$$(\mathbf{x} - \mathbf{p}_0) \cdot \hat{\mathbf{n}} = 0,$$

e a substituição de $\mathbf{r}(t)$ produz a solução linear

$$t = \frac{(\mathbf{p}_0 - \mathbf{o}) \cdot \hat{\mathbf{n}}}{\hat{\mathbf{d}} \cdot \hat{\mathbf{n}}}.$$

A esfera, por sua vez, impõe

$$\|\mathbf{x} - \mathbf{c}\|^2 - r^2 = 0,$$

que, após substituição do raio, leva à quadrática $at^2 + bt + c = 0$, cujo discriminante $\Delta = b^2 - 4ac$ decide ausência, tangência ou dupla interseção [Celes, 2026b, pp. 10–18]. Em ambos os casos, o ponto relevante para a câmera é a menor raiz positiva.

O código espelha esse raciocínio em `Plane.intersect()` e `Sphere.intersect()`. Há

também um segundo cuidado, menos visível na dedução analítica e muito importante na prática: a cena desloca a origem dos raios secundários por um pequeno ε ao longo da normal geométrica. Esse deslocamento evita auto-interseções espúrias em sombras, reflexão e refração, isto é, o clássico *shadow acne*². O ganho aqui não é conceitual, mas numérico: sem essa margem, a formulação teórica correta passa a falhar por erro de ponto flutuante.

2.4 Câmera pinhole, filme e espaços de coordenadas

O Slide 4 reconstrói a câmera pinhole³ por uma base ortonormal local e pela geometria do plano de imagem [Celes, 2026b, pp. 19–39]. Em termos algébricos, isso equivale a uma mudança de base entre espaço da câmera e espaço global. Se θ é o campo de visão vertical e f a distância do pinhole ao plano de projeção, então a meia-altura do filme é dada por

$$\Delta v = f \tan\left(\frac{\theta}{2}\right), \quad \Delta u = \Delta v \frac{w}{h}.$$

Na implementação, `Camera` usa `glm.lookAt()` para construir a transformação de visão e sua inversa, e `generate_ray()` produz o raio primário a partir das coordenadas normalizadas do pixel. A formulação é de câmera pinhole pura: todos os raios partem de um único centro ótico. Isso significa que `focal_distance` é distância geométrica ao plano de projeção, e não distância focal de lente fina nem parâmetro de profundidade de campo. Esse detalhe precisa ser explicitado no texto para que o leitor não projete sobre o código uma física óptica que ele não pretende resolver.

Do lado do filme, `get_sample()` e `get_samples_for_pixel()` deixam claro que o pixel é tratado como domínio de integração. A versão mais simples avalia uma única amostra central; a versão com supersampling⁴ transforma a formação da cor final em uma média sobre várias amostras. A câmera, portanto, fixa a geometria da imagem, enquanto o filme fixa a estratégia de amostragem dessa geometria.

2.5 Phong, luz pontual, termo ambiente e sombras diretas

O fechamento do núcleo do traçador é o modelo local de Phong, expresso como soma de termo ambiente, componente difusa lambertiana e componente especular dependente do vetor refletido [Celes, 2026b, pp. 40–55]. Em forma resumida,

$$L_o \approx k_a L_a + k_d L_i \max(0, \hat{\mathbf{n}} \cdot \hat{\mathbf{l}}) + k_s L_i \max(0, \hat{\mathbf{r}} \cdot \hat{\mathbf{v}})^s.$$

²Artefato numérico de auto-occlusão causado por reinterseção espúria do próprio ponto sombreado depois de erros de arredondamento em ponto flutuante.

³Modelo ideal de formação de imagem em que todos os raios partem de um único centro ótico; por isso, não há abertura finita nem profundidade de campo.

⁴No contexto do projeto, *supersampling* significa estimar a cor do pixel pela média de múltiplas amostras subpixel, e não por filtragem ótica da câmera.

Fisicamente, trata-se de um modelo local e fenomenológico; computacionalmente, ele é suficiente para capturar a dependência angular da superfície sem resolver transporte global de energia. A sombra dura aparece como teste binário de visibilidade entre o ponto sombreado e a fonte.

`PhongMaterial.direct_lighting()` implementa esse cálculo de forma direta: para cada luz, o código obtém radiância incidente e direção, soma a parcela difusa e a parcela especular, e combina o resultado com o termo ambiente. Já `Scene.transmittance()` generaliza o raio de sombra dos slides: para materiais opacos, ele se reduz ao caso binário; para materiais transparentes, ele passa a acumular transmitâncias ao longo do segmento até a fonte. Essa generalização pertence ao bloco mais avançado do projeto, mas sua raiz conceitual já está presente no núcleo do Slide 4.

Há ainda uma nuance importante de modelagem. A formulação didática clássica para fonte pontual costuma incluir decaimento geométrico por $1/r^2$; nesta base, `PointLight` preserva a convenção simplificada do enunciado e trata `power` como radiância incidente constante. Não se trata de erro, mas de simplificação deliberada, pertinente ao contexto didático, e por isso a interpretação física dessa escolha precisa ser declarada explicitamente na seção de limites.

3 Extensões Ópticas, Amostrais e de Transformação

3.1 Antialiasing como integração estocástica sobre o pixel

O Slide 5 reinterpreta o pixel como um domínio de amostragem, e não como uma posição única [Celes, 2026c, pp. 4–8]. A consequência numérica é que a cor do pixel passa a ser estimada pela média de várias amostras de radiância,

$$\hat{L} = \frac{1}{N} \sum_{k=1}^N L(x_k),$$

convertendo aliasing estruturado em ruído de alta frequência. Na base atual, `film.py` opera com três regimes: `center`, `jittered` e `stratified`. A primeira escolha produz o baseline determinístico; as outras duas distribuem as amostras no interior do pixel. Após a refatoração da infraestrutura de amostragem, os padrões bidimensionais passaram a viver em `sampling.py`, o que torna a separação conceitual mais clara: o módulo de amostragem define padrões no quadrado unitário; o filme interpreta esses padrões como coordenadas físicas no pixel.

3.2 Instanciação, coordenadas homogêneas e transformação de normais

O bloco de transformações do Slide 5 introduz a ideia de instância geométrica: em vez de redefinir a interseção de cada forma transformada, o raio é levado para o espaço

local da primitiva canônica [Celes, 2026c, pp. 9–13]. Em coordenadas homogêneas, isso significa aplicar a inversa da transformação ao raio incidente. Para as normais, porém, a transformação correta é a inversa transposta,

$$\mathbf{n}' = (M^{-1})^T \mathbf{n},$$

pois a normal deve permanecer ortogonal ao plano tangente após transformações afins gerais.

`Instance.intersect()` codifica exatamente esse fluxo. O raio entra no espaço local pela matriz inversa; a interseção é resolvida na forma base; a posição volta ao espaço global pela transformação direta; e a normal é convertida pela inversa transposta. Essa é uma daquelas partes do projeto em que a álgebra linear deixa de ser apenas pano de fundo e passa a determinar a correção geométrica do resultado visual.

3.3 Luz de área, integração sobre o emissor e penumbra

Nos slides, a luz de área substitui a fonte pontual idealizada por um emissor com extensão geométrica finita [Celes, 2026c, pp. 14–23]. A penumbra surge porque diferentes sub-regiões da fonte são visíveis ou ocluídas de modo diferente a partir do ponto sombreado. Em termos numéricos, isso exige integrar a contribuição luminosa sobre a superfície emissiva.

`AreaLight.sample_radiance()` aproxima essa integral por soma discreta sobre uma malha `samples_u × samples_v`, com padrões `uniform`, `regular` e `stratified`. Diferentemente da `PointLight`, a fonte extensa introduz atenuação geométrica explícita com a distância. A implementação, portanto, preserva a convenção simplificada da luz pontual, mas adota uma leitura mais geométrica quando a fonte possui área. Essa dualidade precisa aparecer no texto porque ela tem efeito direto na interpretação física das imagens.

3.4 De traçador local a traçador recursivo

O salto conceitual do Slide 5 é que o pipeline local do Slide 4 não é descartado; ele é reutilizado recursivamente [Celes, 2026c, pp. 26–35]. Em termos algorítmicos, alguns materiais deixam de devolver apenas uma cor local e passam a disparar novos raios. Em termos físicos, isso equivale a acrescentar ao estimador alguns caminhos óticos privilegiados: reflexão especular, transmissão especular e atenuação ao longo do trajeto.

Na base atual, `Scene.trace_ray()` mantém-se intencionalmente enxuta: encontra o *hit* mais próximo e delega a avaliação ao material. A profundidade é controlada por `Scene.can_spawn_ray()`, o que impõe um limite finito à recursão. Esse truncamento é necessário para conter o custo computacional e, ao mesmo tempo, coerente com a ideia de que contribuições sucessivas tendem a decair com a refletância ou a transmitância do material.

3.5 Reflexão especular com Fresnel-Schlick

No bloco de reflexão, os slides introduzem a aproximação de Schlick⁵,

$$R(\theta) = R_0 + (1 - R_0)(1 - \cos \theta)^5,$$

como forma de baixo custo para modular a refletância com o ângulo de visão [Celes, 2026c, pp. 26–28]. Na implementação, `ReflectiveMaterial` reaproveita o sombreamento local de Phong e o pondera por $(1 - R)$, enquanto a contribuição do raio refletido é ponderada por R . O resultado é um material recursivo plausível e incrementalmente consistente com o restante do projeto, ainda que não equivalente a um BRDF⁶ fisicamente completo.

3.6 Refração dielétrica, Lei de Snell e Beer-Lambert

O bloco de refração é o trecho matematicamente mais rico do projeto. A interface entre meios obedece à Lei de Snell,

$$\eta_i \sin \theta_i = \eta_t \sin \theta_t,$$

e a intensidade transmitida sofre atenuação com a distância percorrida no meio, segundo a Lei de Beer-Lambert⁷. A implementação traduz essa sequência em três decisões: calcular a fração refletida por Schlick, escolher a razão η_i/η_t com base na orientação da interface e, por fim, atenuar o raio transmitido conforme o comprimento percorrido no material.

`TransparentMaterial.eval()` codifica exatamente essa cadeia. A distinção entre `front_face` e `backfacing` define se o raio entra ou sai do meio; `glm.refract()` produz a direção transmitida; e a reflexão interna total aparece quando a solução refratada se torna inviável. Em paralelo, `TransparentMaterial.shadow_transmittance()` aplica a atenuação de Beer-Lambert aos raios de sombra quando estes deixam o meio. O resultado é uma aproximação consistente para um dielétrico homogêneo em RGB, mas ainda distante de um tratamento espectral completo.

3.7 Da sombra binária à transmitância acumulada

O caso mais elegante de reaproveitamento arquitetural no projeto é a generalização do raio de sombra. O que antes respondia apenas *visível* ou *ocluído* passa a acumular fatores multiplicativos ao atravessar materiais transparentes [Celes, 2026c, pp. 33–35],

⁵Aproximação polinomial de baixo custo para o termo de Fresnel, usada aqui para preservar a dependência angular da refletância sem resolver toda a ótica eletromagnética do problema.

⁶*Bidirectional Reflectance Distribution Function*. O termo aparece aqui apenas como referência conceitual: a implementação preserva a dependência angular da refletância, mas não se propõe a cobrir um modelo fisicamente completo.

⁷Modelo de absorção exponencial no meio, em que a transmitância decai com a distância percorrida e com o coeficiente de absorção.

$$T = \prod_k a_k^{s_k}.$$

Em termos computacionais, `Scene.transmittance()` percorre o segmento até a fonte, interrompe quando encontra um material opaco e, para materiais transparentes, delega ao método `shadow_transmittance()` a atualização do `throughput`⁸. Esse desenho mostra que o projeto não criou um caminho paralelo para transparência; ele estendeu o mecanismo de visibilidade já existente.

4 Triângulos e Estruturas de Aceleração

4.1 Triângulo, área orientada e coordenadas baricêntricas

O Slide 6 começa pelo triângulo como primitiva elementar de malhas [Celes, 2026a, pp. 3–4]. Com vértices **a**, **b** e **c**, os vetores de aresta

$$\mathbf{e}_1 = \mathbf{b} - \mathbf{a}, \quad \mathbf{e}_2 = \mathbf{c} - \mathbf{a}$$

definem uma área orientada

$$A = \frac{\|\mathbf{e}_1 \times \mathbf{e}_2\|}{2},$$

enquanto qualquer ponto interno pode ser descrito por coordenadas baricêntricas,

$$\mathbf{p} = \mathbf{a} + u\mathbf{e}_1 + v\mathbf{e}_2, \quad u \geq 0, v \geq 0, u + v \leq 1.$$

Na base atual, `Triangle` pré-computa **e1**, **e2** e a normal geométrica orientada por produto vetorial. Triângulos degenerados são rejeitados já na construção da forma, o que evita que a malha carregue primitivas nulas para o estágio de interseção.

4.2 Interseção raio-triângulo por Möller-Trumbore

O coração algébrico do Slide 6 é o teste de Möller-Trumbore⁹. Em forma compacta, ele resolve a igualdade

$$\mathbf{o} + t\hat{\mathbf{d}} = \mathbf{a} + u\mathbf{e}_1 + v\mathbf{e}_2,$$

sem construir explicitamente o plano do triângulo. A solução só é válida se $u \geq 0$, $v \geq 0$, $u + v \leq 1$ e $t > 0$. `Triangle.intersect()` implementa exatamente essa forma operacional por meio de `pvec`, `det`, `u`, `v`, `qvec` e `t_candidate`. O teste para `abs(det) < eps` cobre o caso de quase paralelismo entre o raio e o plano do triângulo.

⁸Aqui, *throughput* significa o fator multiplicativo acumulado ao longo do caminho de sombra, e não um fluxo radiométrico absoluto.

⁹Algoritmo direto para interseção raio-triângulo que resolve t , u e v sem construir explicitamente a equação do plano do triângulo.

4.3 Malha triangulada e BVH local

Os slides passam do triângulo isolado para a malha e, em seguida, para estruturas de aceleração [Celes, 2026a, pp. 13–36]. O repositório segue esse mesmo encadeamento. `TriangleMesh` pode ser construído a partir de listas de vértices e faces e participa do mesmo pipeline de interseção, material e iluminação das primitivas analíticas. Quando a malha ativa sua BVH local, caixas AABB¹⁰ passam a funcionar como volumes de contenção, e a travessia recursiva decide em que subárvores vale a pena continuar.

O ponto mais importante aqui é o alcance da aceleração. A base reúne uma BVH estática por malha, construída por mediana em eixo dominante, mas não um acelerador global da cena. Em consequência, a redução de custo aparece dentro da `TriangleMesh`; o topo da cena continua a ser percorrido de forma sequencial por `Scene.compute_intersection()`.

5 Evidências Visuais e Leitura das Figuras

As figuras desta seção usam cenas dedicadas do projeto e algumas extensões selecionadas. O objetivo não é substituir a análise teórica desenvolvida nas seções precedentes, mas oferecer evidência visual organizada do que ela descreve. Em cada caso, a pergunta orientadora é a mesma: o que exatamente a figura mostra, qual comando a reproduz e que hipótese técnica ela ajuda a sustentar. Para preservar a legibilidade no corpo em duas colunas, as imagens ficam contidas na largura da coluna; as versões ampliadas das mesmas evidências aparecem no Anexo C.

5.1 CLI, snapshots e estimativa como infraestrutura experimental

Antes da leitura figura a figura, vale explicitar que os comandos impressos sob cada imagem não são ornamentos editoriais. A camada de CLI centraliza, em `CommonRenderOptions`, os parâmetros que realmente controlam o experimento – resolução, `spp`, `sampling_mode`, semente e opções de calibração – e os redistribui tanto para os `entrypoints` quanto para o estimador. Isso impede que cada script de cena redefina sua própria interface e garante que comparações como as de `req2`, `req4`, `proj1_final` e `proj1_final_v2` diferenciem a cena ou o regime de amostragem, e não o contrato de execução.

Essa padronização só se torna metodologicamente útil porque cada render também gera um snapshot persistente. Em `RenderSnapshot.from_runtime()`, o estado do render é serializado em termos de configuração de imagem, câmera, cena, objetos e luzes, e depois gravado em `properties.json` e `properties.md`. Com isso, exemplos potencialmente dispersos voltam a ser legíveis como evidência comparável: `proj1_final` e `proj1_final_v2` preservam `800 × 600`, `sampling_mode=jittered`, `spp=1`, `max_depth=8`

¹⁰ *Axis-Aligned Bounding Boxes*: volumes de contenção alinhados aos eixos cartesianos, usados aqui para poda espacial local.



Figura 1. Cena de geometria e instanciação: dois `Instance(Box)` e uma `Sphere` sob câmera fixa e iluminação ambiente (`800×600`, `spp=1`).

Comando gerador

```
python -m ray_tracing_2.proj1_req1_geometry --width
800 --height 600 --sampling_mode center --spp 1
```

e a mesma câmera, de modo que a diferença de custo entre `144,2s` e `172,0s` não pode ser atribuída a uma configuração accidental. No extremo oposto, `proj1_heart_trianglemesh` registra explicitamente `spp=25`, `sampling_mode=jittered`, acelerador `bvh` e tempo real de `1008,0s`, o que preserva o contexto técnico de uma imagem custosa que, isolada, seria difícil de interpretar.

A terceira peça é o estimador. `RenderEstimator.maybe_calibrate()` mede rapidamente o throughput em uma grade reduzida, ajusta o modelo de raios e, ao final de `run()`, grava no snapshot o comparativo entre tempo estimado e tempo real. Isso transforma tempo de render em evidência analisável, não apenas em conveniência de linha de comando. Os artefatos mostram que a aproximação é boa em `proj1_heart_trianglemesh`, com estimativa de `1122,4s` para um tempo observado de `1008,0s` e erro relativo de `11,36%`. Já nas cenas integradas `proj1_final` e `proj1_final_v2`, o estimador prevê `63,6s` e `74,3s`, mas o render efetivo consome `144,2s` e `172,0s`, isto é, cerca de $2,3\times$ mais. Longe de enfraquecer a análise, essa diferença explicita que o custo cresce rapidamente quando a cena combina sombras, superfícies recursivas e maior densidade de interseções, e mostra por que os snapshots e o estimador são parte do argumento técnico do relatório, e não material periférico.

5.2 Geometria, câmera e instanciação

Esta cena é a melhor porta de entrada para a leitura do núcleo geométrico. Ela preserva uma câmera fixa e troca complexidade fotométrica por legibilidade espacial. Na Figura 1, o ponto importante não é apenas a coexistência de uma esfera e de caixas, mas a coerência geométrica da instância: a primitiva base é reutilizada com transformação afim, sem reescrever a interseção da caixa no espaço global.

5.3 Iluminação local com múltiplas fontes pontuais

Na Figura 2, a presença de três `PointLight` torna visível a contribuição angular de fontes distintas sobre materiais diferentes. O efeito importante da cena é pedagógico: ela torna fácil perceber que a componente especular não depende apenas da posição da luz, mas também do material e do ponto de vista. As duas esferas com `shininess` diferente cumprem exatamente esse papel.

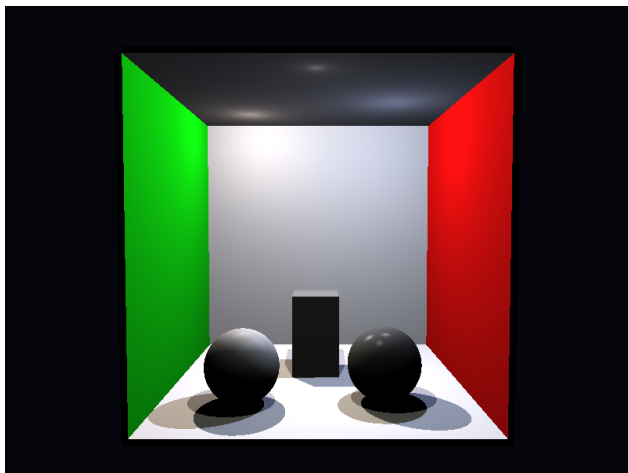


Figura 2. Cena com múltiplas luzes pontuais e variação do brilho especular em materiais com `shininess` distinto no baseline center (800x600, spp=1, tempo de render aproximado de 19,66s).

Comando gerador

```
python -m ray_tracing_2.proj1_req2_point_lights
--width 800 --height 600 --sampling_mode center --spp
1
```

Como exemplo simples de comparação de antialiasing sobre a mesma configuração geométrica e fotométrica, a Figura 3 repete essa cena com `stratified` e 25 amostras por pixel. A diferença visual não altera a leitura física da cena, mas reduz serrilhados e estabiliza bordas e *highlights* especulares ao custo de um crescimento expressivo no número total de raios.

5.4 Phong e sombras diretas

Esta cena foi escolhida para expor o núcleo fotométrico mínimo do projeto. Na Figura 4, a leitura correta é que o mesmo pipeline decide qual superfície foi atingida e, depois, se cada luz contribui ou não para aquele ponto. O contraste entre o vermelho mais fosco e o azul mais brilhante ajuda a separar visualmente o termo difuso da concentração angular do termo especular.

5.5 Amostragem sobre o pixel

Na Figura 5, a geometria foi escolhida para que arestas finas e contrastes fortes deixem o aliasing visível. As três imagens seguintes usam a mesma câmera, a mesma geometria, as mesmas luzes e o mesmo enquadramento; o que muda é apenas a estratégia de amostragem do pixel. O ponto importante não é provar convergência estatística, mas deixar claro que a base já separa câmera, geometria e amostragem do pixel, o que permite

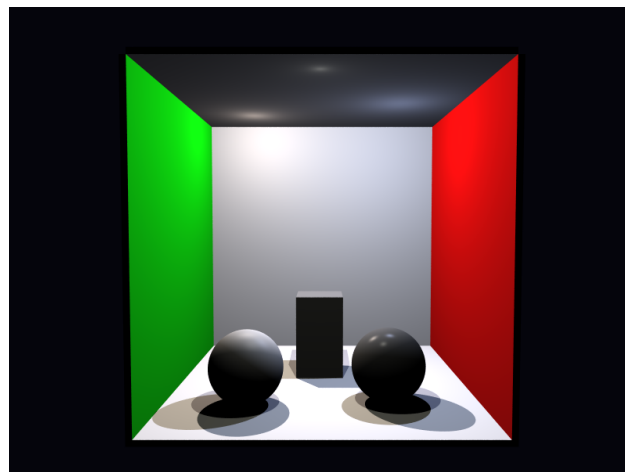


Figura 3. Mesma cena de múltiplas luzes pontuais com `sampling_mode=stratified` e 25 amostras por pixel: melhora a integração espacial sem alterar materiais, luzes ou câmera (800x600, tempo de render aproximado de 501,91s).

Comando gerador

```
python -m ray_tracing_2.proj1_req2_point_lights
--width 800 --height 600 --sampling_mode stratified
--spp 25 --seed 42
```

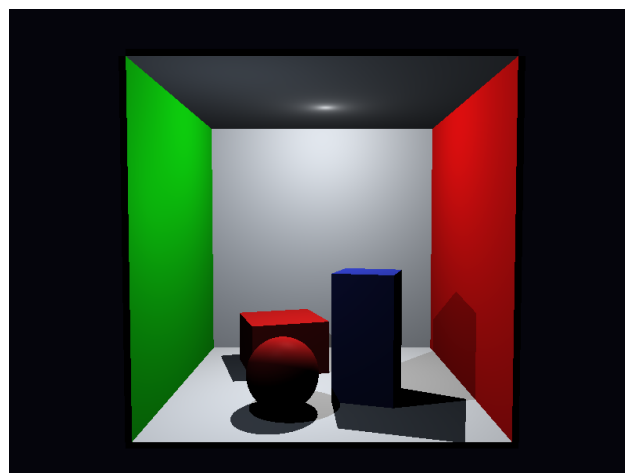


Figura 4. Cena de sobreposição local com Phong e sombras diretas duras por visibilidade até a fonte (800x600, spp=1).

Comando gerador

```
python -m ray_tracing_2.proj1_req3_phong_shadows
--width 800 --height 600 --sampling_mode center --spp
1
```

comparar regimes de integração sem reconfigurar o restante da cena.

Lidas em sequência, as Figuras 5, 6 e 7 mostram exatamente o ganho pedagógico do requisito de amostragem múltipla: `center` preserva o baseline determinístico e barato; `jittered` quebra o padrão regular do aliasing ao espalhá-lo como ruído; e `stratified` cobre o pixel de forma mais uniforme, mantendo custo muito próximo ao do `jittered`, mas com leitura visual mais estável.

5.6 Malha triangular com BVH local

A Figura 8 sintetiza a parte mais importante do bloco de triângulos e aceleração. A evidência principal não é

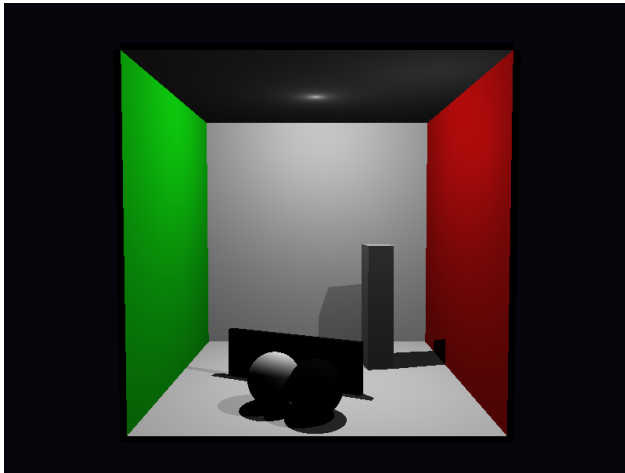


Figura 5. Cena-base para comparação entre *center*, *jittered* e *stratified* no domínio do pixel (800x600, *sampling_mode=center*, *spp=1*, tempo de render aproximado de 20,65s).

Comando gerador

```
python -m ray_tracing_2.proj1_req4_sampling --width
800 --height 600 --sampling_mode center --spp 1
```

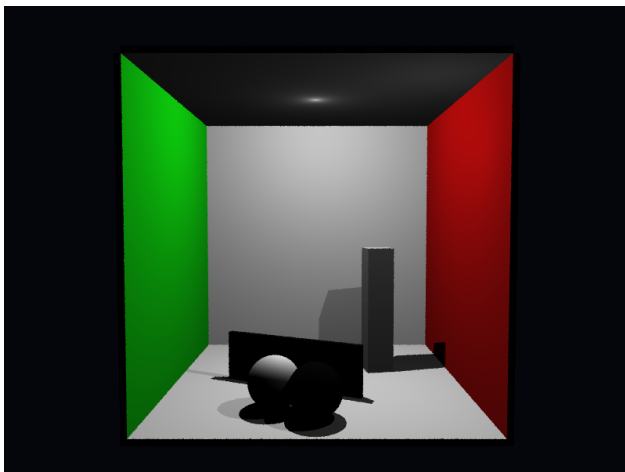


Figura 6. Mesma cena de antialiasing com *sampling_mode=jittered* e 4 amostras por pixel: o serrilhado estruturado vira ruído espacial mais difuso, com tempo de render aproximado de 86,56s.

Comando gerador

```
python -m ray_tracing_2.proj1_req4_sampling --width
800 --height 600 --sampling_mode jittered --spp 4
--seed 42
```

apenas que a malha aparece na imagem, mas que ela participa do mesmo pipeline de materiais e luzes e, ao mesmo tempo, é percorrida por uma BVH local. Essa figura deve ser lida em conjunto com a discussão teórica da seção anterior: o ganho é real para a malha, mas não autoriza falar em acelerador global da cena.

5.7 Luz de área e emissor visível

Na Figura 9, a leitura relevante é dupla. Primeiro, a cena mostra que a luz de área produz penumbra por amostragem espacial do emissor. Segundo, ela registra uma decisão de arquitetura que a parte teórica precisa explicar: a luminária visível é obtida por

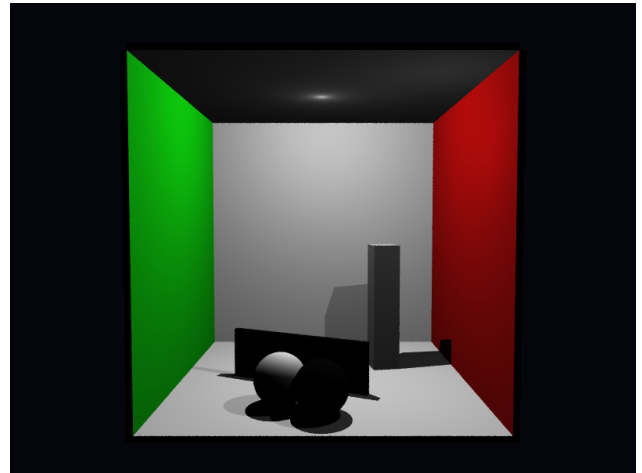


Figura 7. Mesma cena de antialiasing com *sampling_mode=stratified* e 4 amostras por pixel: a cobertura espacial mais uniforme do pixel reduz ruído e estabiliza contornos em relação ao *jittered*, com tempo de render aproximado de 84,84s.

Comando gerador

```
python -m ray_tracing_2.proj1_req4_sampling --width
800 --height 600 --sampling_mode stratified --spp 4
--seed 42
```

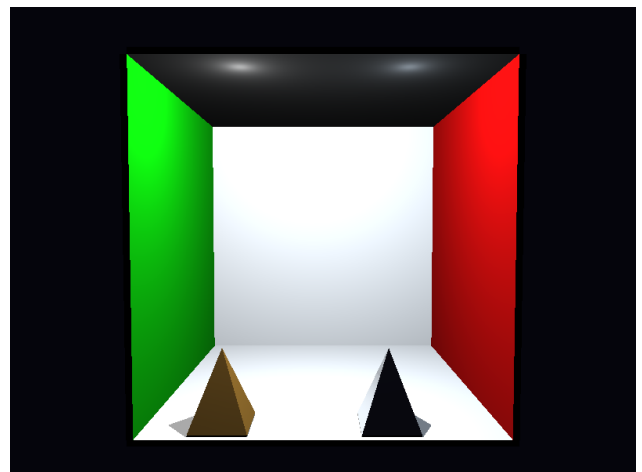


Figura 8. Cena triangular com BVH local: a imagem visualiza a malha sob o mesmo pipeline de iluminação, enquanto a poda por AABB reduz o custo interno da interseção (800x600, *spp=1*).

Comando gerador

```
python -m ray_tracing_2.proj1_rext_bvh --width 800
--height 600 --sampling_mode center --spp 1
```

EmissiveMaterial, enquanto a iluminação direta da cena continua a vir da *AreaLight*. Em outras palavras, o painel aceso visto pela câmera e a fonte utilizada para *shadow rays* não são a mesma camada de abstração.

5.8 Refração, TIR e Beer-Lambert

A Figura 10 concentra a parte óptica mais avançada da base. A cena não prova isoladamente cada equação, mas é a melhor evidência integrada de que direção refratada, reflexão interna total e atenuação no interior do meio passaram a participar do mesmo fluxo de visibilidade e recursão. O ponto metodológico importante é este: o resultado visual não aparece como módulo separado, e

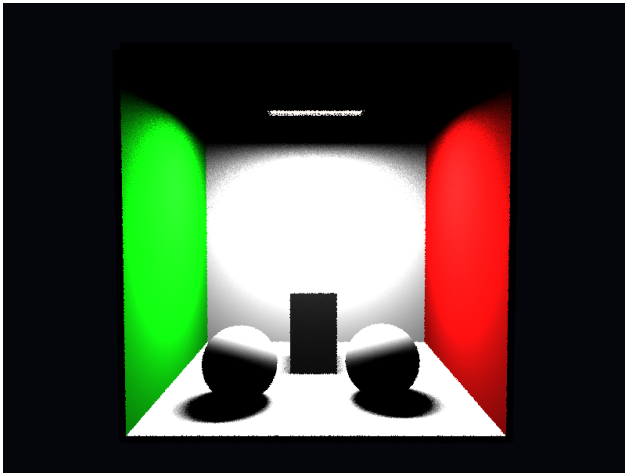


Figura 9. Cena com luz de área retangular e emissor visível: a penumbra vem da fonte extensa amostrada, enquanto a aparência luminosa do painel vem de `EmissiveMaterial` (800x600, spp=1).

Comando gerador

```
python -m ray_tracing_2.proj1_rext_area_light --width 800 --height 600 --sampling_mode center --spp 1
```

sim como extensão do mesmo traçador que já tratava Phong e sombras diretas.

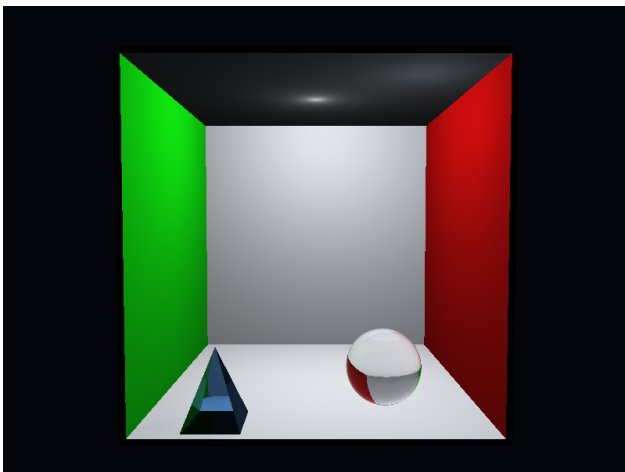


Figura 10. Cena com material refratário: o resultado combina Lei de Snell, reflexão interna total e atenuação tipo Beer-Lambert dentro do mesmo pipeline recursivo (800x600, spp=1).

Comando gerador

```
python -m ray_tracing_2.proj1_rext_refractive --width 800 --height 600 --sampling_mode center --spp 1
```

5.9 AA em uma cena integrada

Uma segunda comparação, agora em uma cena mais rica, ajuda a verificar se a leitura feita para `req4` persiste quando há mais oclusões, profundidade visual e superfícies com maior contraste local. As Figuras 11 e 12 usam a mesma *Cornell box* com pirâmide interna; o ganho do `stratified` continua qualitativo, mas aparece agora em uma composição mais integrada, na qual o custo do render cresce de aproximadamente 49,76s para 200,80s.

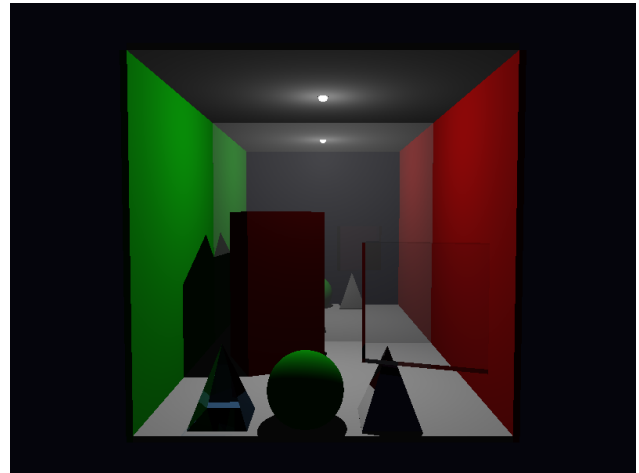


Figura 11. Cena integrada *Cornell box* com pirâmide no baseline center: referência para avaliar como aliasing e ruído aparecem em uma composição mais rica (800x600, spp=1, tempo aproximado de 49,76s).

Comando gerador

```
python -m ray_tracing_2.cornell_box_pyramid --width 800 --height 600 --sampling_mode center --spp 1
```

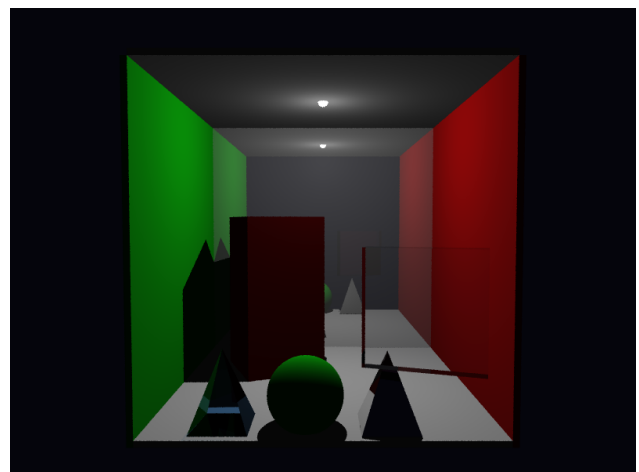


Figura 12. Mesma cena integrada com `sampling_mode=stratified` e 4 amostras por pixel: a melhoria observada em `req4` reaparece em uma imagem mais carregada, ao custo de aproximadamente 200,80s.

Comando gerador

```
python -m ray_tracing_2.cornell_box_pyramid --width 800 --height 600 --sampling_mode stratified --spp 4 --seed 42
```

5.10 Cenas integradas recuperadas por snapshots

Os exemplos anexados de cenas finais merecem aparecer também como evidência visual, e não apenas como números citados na subseção de infraestrutura experimental. As Figuras 13, 14 e 15 mostram três casos em que os snapshots preservam não só a imagem, mas o contexto técnico necessário para interpretá-la: nas duas cenas finais, a câmera e o regime básico de render permanecem estáveis, o que torna comparável a diferença entre 144,2s e 172,0s; na cena da malha cardíaca, o snapshot registra explicitamente `spp=25`,

`sampling_mode=jittered` e acelerador `bvh`, o que explica por que o custo sobe para 1008,0 s sem transformar a imagem em um caso opaco de alto tempo de render.

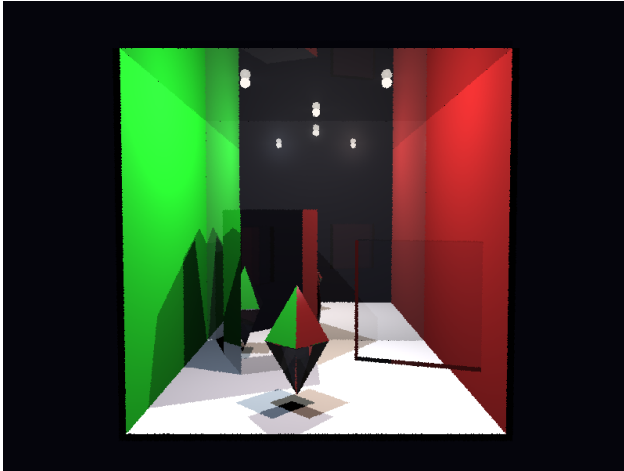


Figura 13. Cena final integrada `proj1_final`: combinação de materiais reflexivos, refratários e malhas triangulares em um caso no qual o estimador prevê 63,6 s, mas o render observado chega a 144,2 s.

Comando gerador

```
python -m ray_tracing_2.proj1_final --width 800
--height 600 --spp 1
```

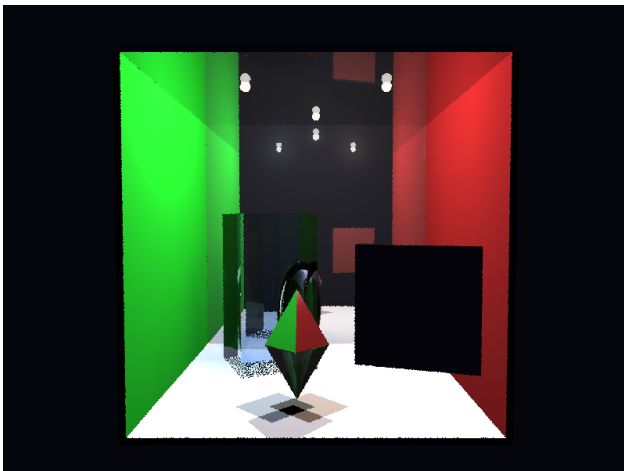


Figura 14. Variante `proj1_final_v2`: mesma leitura de cena integrada sob câmera equivalente, agora com tempo real de 172,0 s para uma previsão de 74,3 s, reforçando a sensibilidade do custo à composição geométrica e material.

Comando gerador

```
python -m ray_tracing_2.proj1_final_v2 --width 800
--height 600 --spp 1
```

Os demais artefatos exploratórios recuperados via snapshot – da cena mínima `main_scene` a variações sucessivas de `cornell_box` e a uma versão adicional de `cornell_box_pyramid` – ficam reunidos no Anexo C. Eles não são centrais para a progressão teórica do corpo principal, mas ajudam a mostrar que a instrumentação do projeto preservou evidência útil também fora do conjunto canônico de cenas escolhido para a narrativa prin-



Figura 15. Cena `proj1_heart_trianglemesh`: malha triangulada com `bvh` e 25 amostras por pixel, em um caso onde a estimativa 1122,4 s permanece relativamente próxima do tempo observado 1008,0 s.

Comando gerador

```
python -m ray_tracing_2.proj1_heart_trianglemesh
--width 800 --height 600 --spp 25 --sampling_mode
jittered
```

cipal.

6 Decisões de Modelagem e Limites

6.1 Convenções radiométricas

O projeto não usa a mesma convenção radiométrica para todas as fontes. `PointLight` preserva a interpretação simplificada do enunciado e trata `power` como radiação incidente constante, sem fator explícito de decaimento por $1/r^2$. Já `AreaLight` distribui energia pelas amostras do emissor e introduz atenuação geométrica com a distância. Essa heterogeneidade é aceitável no contexto didático do Projeto 1, mas precisa ser declarada para evitar leitura física mais forte do que a base sustenta.

6.2 Câmera e formação de imagem

Toda a formação de imagem permanece no modelo pinhole [Celes, 2026b, pp. 19–39]. Isso significa que `focal_distance` controla apenas a geometria do plano de projeção. Não há lente fina, abertura finita nem profundidade de campo. O supersampling melhora a integração sobre o pixel, mas não muda a física da câmera. Portanto, as imagens devem ser lidas como resultados de câmera pinhole com amostragem estocástica, e não como simulações ópticas completas.

6.3 Aceleração e complexidade de cena

A parte de triângulos e aceleração cobre apenas uma fração do arco teórico da disciplina. Há BVH estática por `TriangleMesh`, com AABB e travessia recursiva simplificada, mas não há grade regular, SAH¹¹, BVH linearizada nem estrutura global de aceleração. Em termos de custo, isso significa que a interseção no topo da cena

¹¹ *Surface Area Heuristic*, heurística clássica para particionamento de BVHs que não é implementada nesta base.

continua linear no número de objetos, e o ganho de desempenho aparece apenas dentro das malhas trianguladas.

6.4 Materiais recursivos e truncamento

Os materiais reflexivos e refratários ampliam bastante o escopo do projeto, mas continuam sendo aproximações controladas. A recursão é truncada por `max_depth`, o que contém o custo computacional e evita crescimento explosivo do número de raios, ao preço de interromper cadeias longas de reflexão e transmissão. Da mesma forma, a combinação entre Phong local, Fresnel-Schlick, Snell, TIR e Beer-Lambert produz uma ótica geometricamente rica, mas ainda distante de um solucionador completo de transporte global de luz.

7 Conclusão

Tomando o estado atual do repositório como objeto de análise, o projeto reúne um núcleo geométrico e fotométrico sólido: raio paramétrico, interseções analíticas, câmera pinhole, filme com suporte a supersampling, sombreamento local de Phong e sombras diretas. Sobre esse núcleo, a base acrescenta extensões relevantes do conteúdo da disciplina, como instanciamento afim, luz de área, reflexão especular com Fresnel-Schlick, refração dielétrica com Lei de Snell, Beer-Lambert e uma BVH local para malhas triangulares.

O mérito principal do relatório é tornar essa leitura tecnicamente defensável sem perder fluidez. A parte teórica permanece como corpo do texto, as imagens ocupam um espaço próprio de interpretação acompanhado por comandos reproduzíveis, snapshots persistentes e estimativas de custo, e a rastreabilidade é consolidada em quadros finais que ligam conceitos de referência, evidência concreta e notas com páginas dos slides e linhas de código. Assim, o documento mantém densidade analítica sem fragmentar cada subseção com blocos locais de rastreabilidade.

Em síntese, o repositório entrega um ray tracer educacional reproduzível, com fronteiras de escopo relativamente bem definidas. O que falta está concentrado em modelagem física mais rigorosa e em estruturas globais de aceleração, e não em inconsistência do pipeline principal. Essa distinção é importante porque permite avaliar o trabalho com precisão: não como cobertura integral do arco teórico das aulas, mas como realização coerente, incremental e tecnicamente bem localizada de uma parcela significativa desse arco.

Referências

Celes, W. (2026a). Estruturas de aceleração. Slides de aula em PDF da disciplina Fundamentos de Computação Gráfica, hospedados no GitHub. Autor dos slides: W. Celes. Contato: celes@inf.puc-rio.br. Disponível em: https://github.com/yangricardo/INF2608-comp-grafica/blob/main/materiais/tra%C3%A7ado_de_raios/6.estrutura_aceleracao.pdf. Acesso em: 1 maio 2026.

Celes, W. (2026b). Traçado de raios. Slides de aula em PDF da disciplina Fundamentos de Computação Gráfica, hospedados no GitHub. Autor dos slides: W. Celes. Contato: celes@inf.puc-rio.br. Disponível em: https://github.com/yangricardo/INF2608-comp-grafica/blob/main/materiais/tra%C3%A7ado_de_raios/4.tracado_de_raios.pdf. Acesso em: 1 maio 2026.

Celes, W. (2026c). Traçado de raios – parte ii. Slides de aula em PDF da disciplina Fundamentos de Computação Gráfica, hospedados no GitHub. Autor dos slides: W. Celes. Contato: celes@inf.puc-rio.br. Disponível em: https://github.com/yangricardo/INF2608-comp-grafica/blob/main/materiais/tra%C3%A7ado_de_raios/5.tracado_de_raios2.pdf. Acesso em: 1 maio 2026.

Miranda, Y. R. B. (2026). Ray tracer educacional do projeto 1 de inf2608 – fundamentos de computação gráfica. Repositório GitHub do projeto. Disponível em: <https://github.com/yangricardo/INF2608-comp-grafica>. Acesso em: 1 maio 2026.

Pharr, M., Jakob, W., and Humphreys, G. (2023). *Physically Based Rendering: From Theory to Implementation*. MIT Press, 4 edition. Online edition: <https://pbr-book.org/4ed/contents>.

Anexo A: Quadros Consolidados de Referência e Evidência

Os quadros a seguir sintetizam a ligação entre os conceitos de referência discutidos no corpo do texto e a evidência mais direta encontrada na base. A coluna *Nota* registra, em forma compacta, as páginas dos slides, as referências externas mais próximas e as linhas de código mais relevantes para cada conceito.

Núcleo geométrico e fotométrico
Extensões ópticas, amostrais e de transformação
Infraestrutura experimental e reprodutibilidade
Geometria triangular e aceleração local

Tabela 1. Quadro consolidado do núcleo geométrico e fotométrico.

Conceito de referência	Evidência principal	Nota
Raio paramétrico, visibilidade e registro de hit	<code>Ray</code> , <code>Hit</code> , <code>Scene.compute_intersection()</code> , <code>Scene.trace_ray()</code>	Slides 04, pp. 7–9; PBRT 4e, caps. 1.2 e 3.5; <code>hit.py</code> :11, 25; <code>scene.py</code> :33, 137.
Interseções raio-plano e raio-esfera	<code>Sphere.intersect()</code> , <code>Plane.intersect()</code> , deslocamento por ε em <code>offset_point()</code>	Slides 04, pp. 10–18; <code>shape.py</code> :17, 23, 54, 60; <code>scene.py</code> :50.
Câmera pinhole, filme e geração de raios primários	<code>Camera.generate_ray()</code> , <code>Film.get_sample()</code> , <code>Film.get_samples_for_pixel()</code> , <code>Film.render()</code>	Slides 04, pp. 19–39; PBRT 4e, cap. 5.2; <code>camera.py</code> :6, 28; <code>film.py</code> :74, 98, 104, 156.
Modelo local de Phong, ambiente e sombras diretas	<code>PhongMaterial.direct_lighting()</code> , <code>PhongMaterial.eval()</code> , <code>PointLight.radiance()</code> , <code>Scene.transmittance()</code>	Slides 04, pp. 40–55; <code>material.py</code> :70, 79, 106; <code>light.py</code> :51, 55; <code>scene.py</code> :66.
Convenção simplificada para fonte pontual	<code>PointLight.radiance()</code> sem fator explícito de $1/r^2$	Slides 04, pp. 40–55; escolha didática coerente com o enunciado; <code>light.py</code> :51, 55.
Pipeline câmera–interseção–material	<code>Film.render()</code> , <code>Camera.generate_ray()</code> , <code>Scene.compute_intersection()</code> , <code>Scene.trace_ray()</code>	Slides 04, pp. 44–55; <code>film.py</code> :156; <code>camera.py</code> :28; <code>scene.py</code> :33, 137.

Tabela 2. Quadro consolidado das extensões ópticas, amostrais e de transformação.

Conceito de referência	Evidência principal	Nota
Antialiasing por múltiplas amostras no pixel	<code>uniform_samples_2d()</code> , <code>stratified_grid_samples_2d()</code> , <code>Film.get_samples_for_pixel()</code>	Slides 05, pp. 4–8; PBRT 4e, cap. 8; <code>sampling.py</code> :15, 27, 43; <code>film.py</code> :104, 156.
Instanciação afim e transformação correta de normais	<code>Instance.intersect()</code> com uso de inversa e inversa transposta	Slides 05, pp. 9–13; PBRT 4e, cap. 3.5; <code>shape.py</code> :253, 262.
Luz de área, amostragem do emissor, penumbra e emissor visível	<code>AreaLight.sample_radiance()</code> , <code>AreaLight.radiance()</code> , <code>EmissiveMaterial.eval()</code>	Slides 05, pp. 14–23; PBRT 4e, cap. 12.4; <code>light.py</code> :80, 135, 195; <code>material.py</code> :37, 50.
Caixa, AABB e método de slabs	<code>Box.intersect()</code> , <code>AABB.intersects()</code>	Slides 05, pp. 24–25; PBRT 4e, cap. 3.7; <code>shape.py</code> :74, 88; <code>triangle_bvh.py</code> :50, 65.
Traçador recursivo com profundidade finita	<code>Scene.can_spawn_ray()</code> , <code>Scene.trace_ray()</code>	Slides 05, pp. 26–35; <code>scene.py</code> :59, 137.
Reflexão especular com Fresnel-Schlick	<code>ReflectiveMaterial.eval()</code>	Slides 05, pp. 26–28; PBRT 4e, cap. 9.3; <code>material.py</code> :118, 136.
Refração dielétrica, TIR e Beer-Lambert	<code>TransparentMaterial.eval()</code> , <code>shadow_transmittance()</code>	Slides 05, pp. 29–35; PBRT 4e, caps. 9.5 e 11.2; <code>material.py</code> :176, 206, 227.
Transmitância acumulada em raios de sombra	<code>Scene.transmittance()</code> e delegação para <code>shadow_transmittance()</code>	Slides 05, pp. 33–35; <code>scene.py</code> :66; <code>material.py</code> :30, 61, 206.

Tabela 3. Quadro consolidado da infraestrutura experimental e da reprodutibilidade.

Conceito de referência	Evidência principal	Nota
CLI padronizada para execuções comparáveis	<code>CommonRenderOptions</code> e conversão uniforme de parâmetros para <i>entrypoints</i> e estimador	Camada de engenharia que estabiliza resolução, <code>spp</code> , <code>sampling_mode</code> , semente e calibração; <code>cli.py</code> :16, 50, 64, 96, 174.
Snapshot persistente do estado do render	<code>RenderSnapshot.from_runtime()</code> e serialização em <code>properties.json/properties.md</code>	Registra render, câmera, cena, objetos e luzes em artefatos auditáveis; <code>render_snapshot.py</code> :524, 605, 656, 665, 770, 773.
Estimativa, calibração e comparação com o tempo real	<code>RayCountEstimator</code> e execução coordenada por <code>RenderEstimator</code>	Mede throughput, atualiza o modelo e persiste o comparativo estimado/real ao fim do render; <code>render_estimator.py</code> :34, 189, 277, 368, 389, 492.

Tabela 4. Quadro consolidado da geometria triangular e da aceleração local.

Conceito de referência	Evidência principal	Nota
Triângulo como primitiva elementar e normal orientada	<code>Triangle</code> , vetores de aresta <code>e1/e2</code> , normal geométrica	Slides 06, pp. 3–4; <code>shape.py</code> :144.
Interseção raio-triângulo por Möller–Trumbore	<code>Triangle.intersect()</code>	Slides 06, pp. 5–12; <code>shape.py</code> :158.
Malha triangulada integrada ao pipeline geral	<code>TriangleMesh</code> , <code>TriangleMesh.intersect()</code> , <code>Scene.compute_intersection()</code>	Slides 06, pp. 13–14; <code>shape.py</code> :188, 241; <code>scene.py</code> :33.
BVH local para malhas trianguladas	<code>TriangleBVH</code> , <code>TriangleBVHNode.intersect()</code> , <code>TriangleBVH._build()</code>	Slides 06, pp. 22–36; PBRT 4e, cap. 7.3; <code>triangle_bvh.py</code> :104, 115, 146, 160, 198.
AABB como volume de contenção	<code>AABB</code> , <code>AABB.intersects()</code>	Slides 06, pp. 22–36; <code>triangle_bvh.py</code> :50, 65.
Percurso sequencial no topo da cena	<code>Scene.compute_intersection()</code> como evidência da ausência de agregador global	Slides 06, pp. 22–36; <code>scene.py</code> :33.

Anexo B: Diagramas de Classes

Este anexo passa a reunir duas leituras complementares da arquitetura principal do pacote `ray_tracing_2`. A primeira preserva a versão inicial do diagrama, mais compacta e centrada no pipeline geométrico e fotométrico. A segunda, `v3`, estende essa base para incluir a camada de CLI, estimativa/calibração e snapshots, que hoje participa diretamente da reprodutibilidade experimental discutida no corpo do texto.

Versão inicial

Versão complementar estendida (v3)

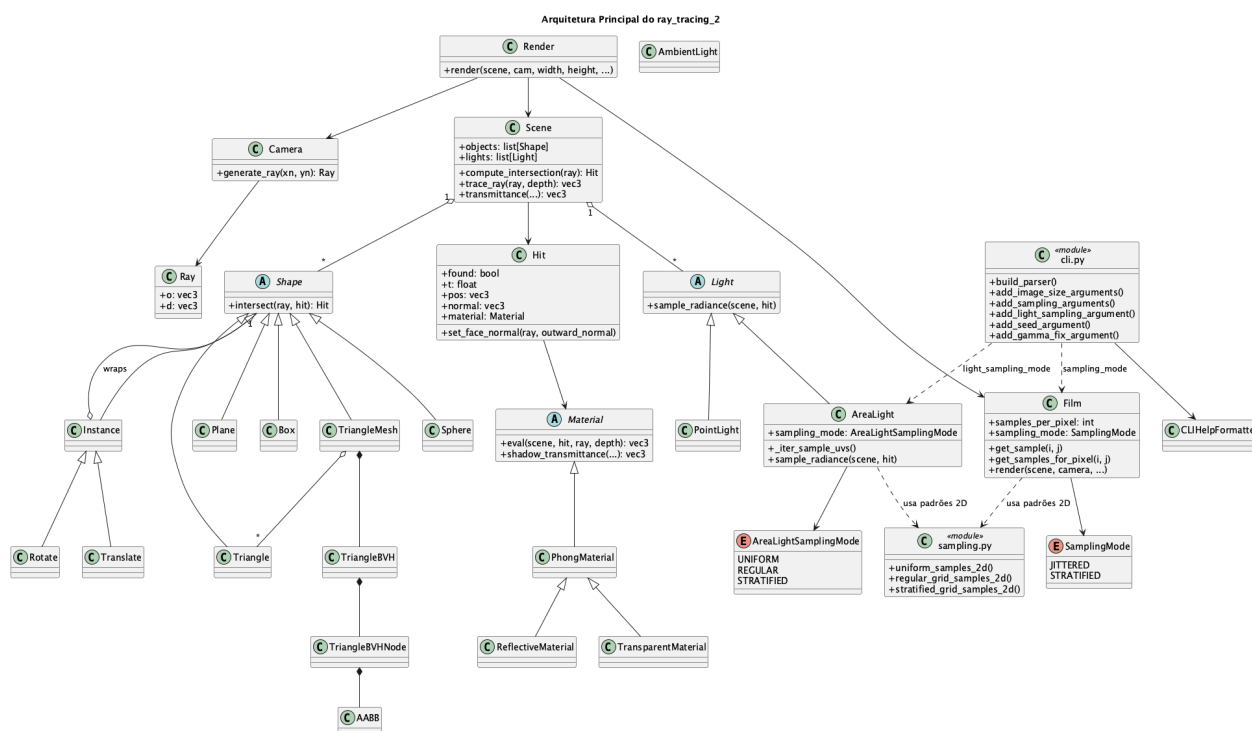


Figura 16. Versão inicial do diagrama de classes de `ray_tracing_2`, focada no pipeline principal de câmera, filme, cena, materiais, luzes e geometria.

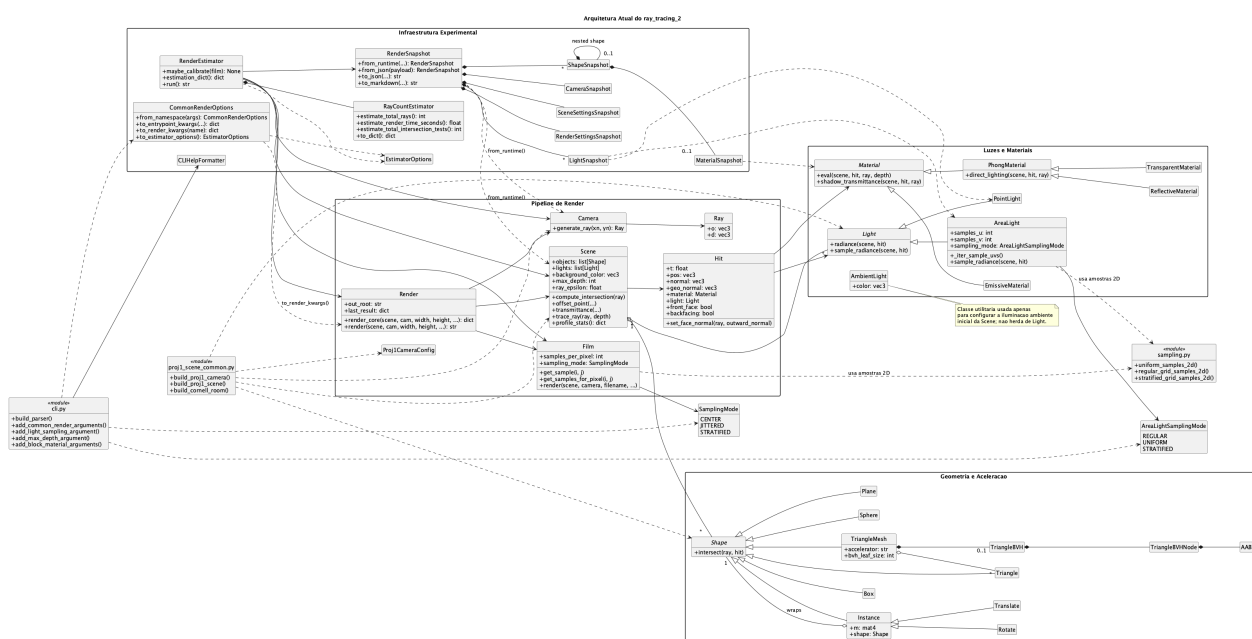


Figura 17. Versão v3 do diagrama de classes de `ray_tracing_2`, adicionando a infraestrutura experimental de CLI, estimativa e snapshots como extensão da leitura inicial.

Anexo C: Galeria Visual Ampliada

As figuras a seguir reproduzem, em escala ampliada, as principais evidências visuais discutidas no corpo do texto. O objetivo deste anexo é preservar a legibilidade das imagens sem comprometer a diagramação do relatório em duas colunas.



Figura 18. Versão ampliada da Figura 1: geometria e instanciação.

Exemplos exploratórios recuperados por snapshot

As figuras a seguir reúnem artefatos exploratórios anexados pelo autor e preservados pelas saídas `properties.md/properties.json`. Elas complementam o corpo do texto ao mostrar como a camada de snapshots permite recuperar cenas intermediárias e variações de modelagem que não entraram na espinha principal do relatório, mas continuam úteis como evidência de exploração técnica.

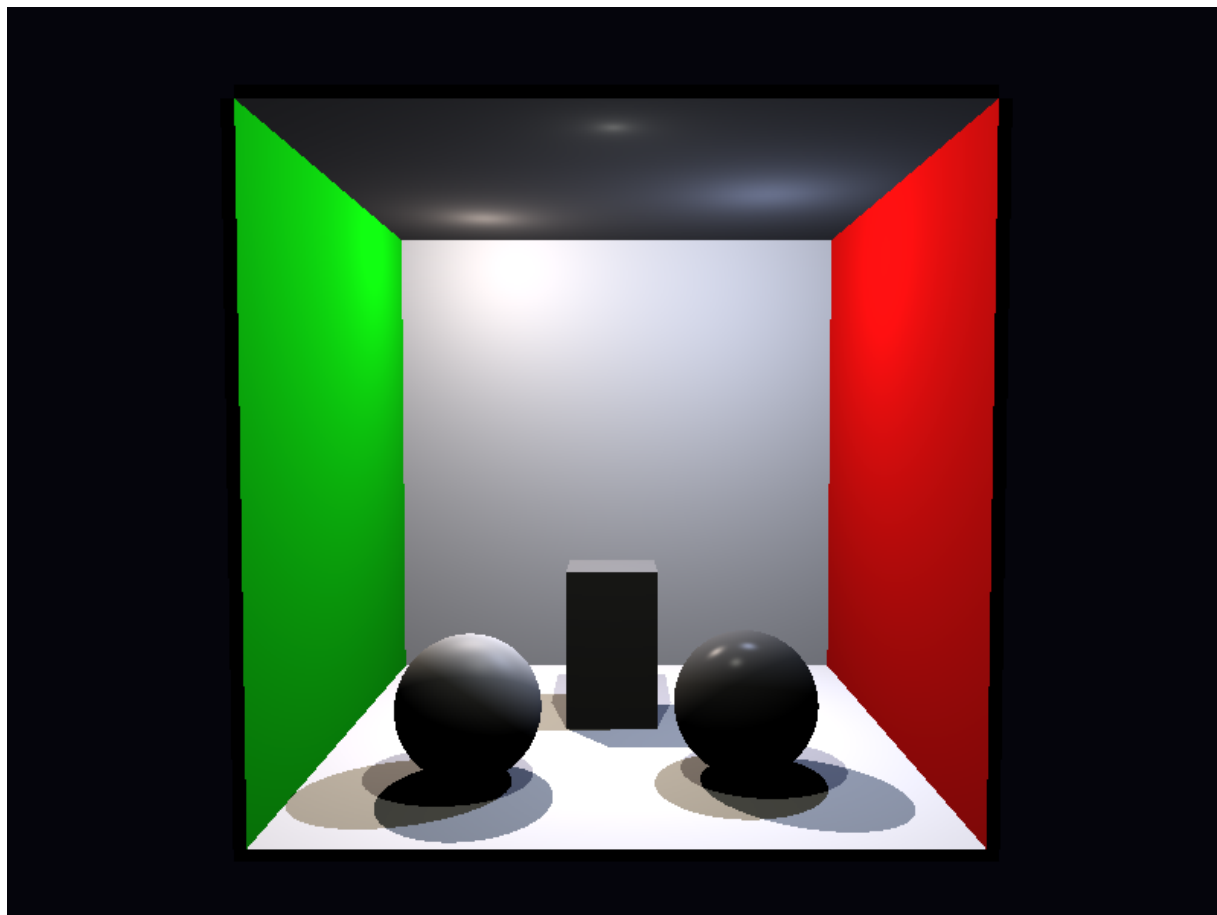


Figura 19. Versão ampliada da Figura 2: múltiplas fontes pontuais no baseline center.

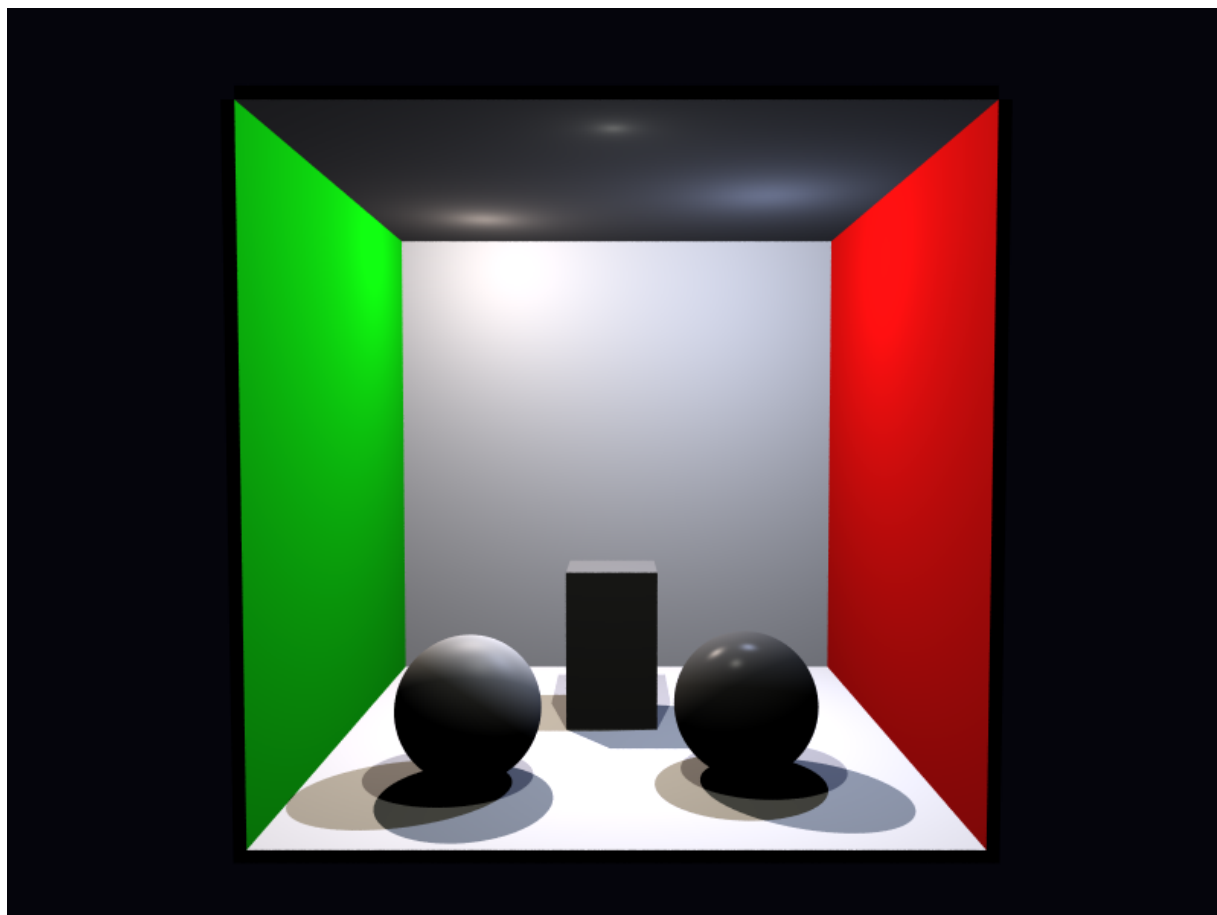


Figura 20. Versão ampliada da Figura 3: mesma cena com stratified e 25 amostras por pixel.

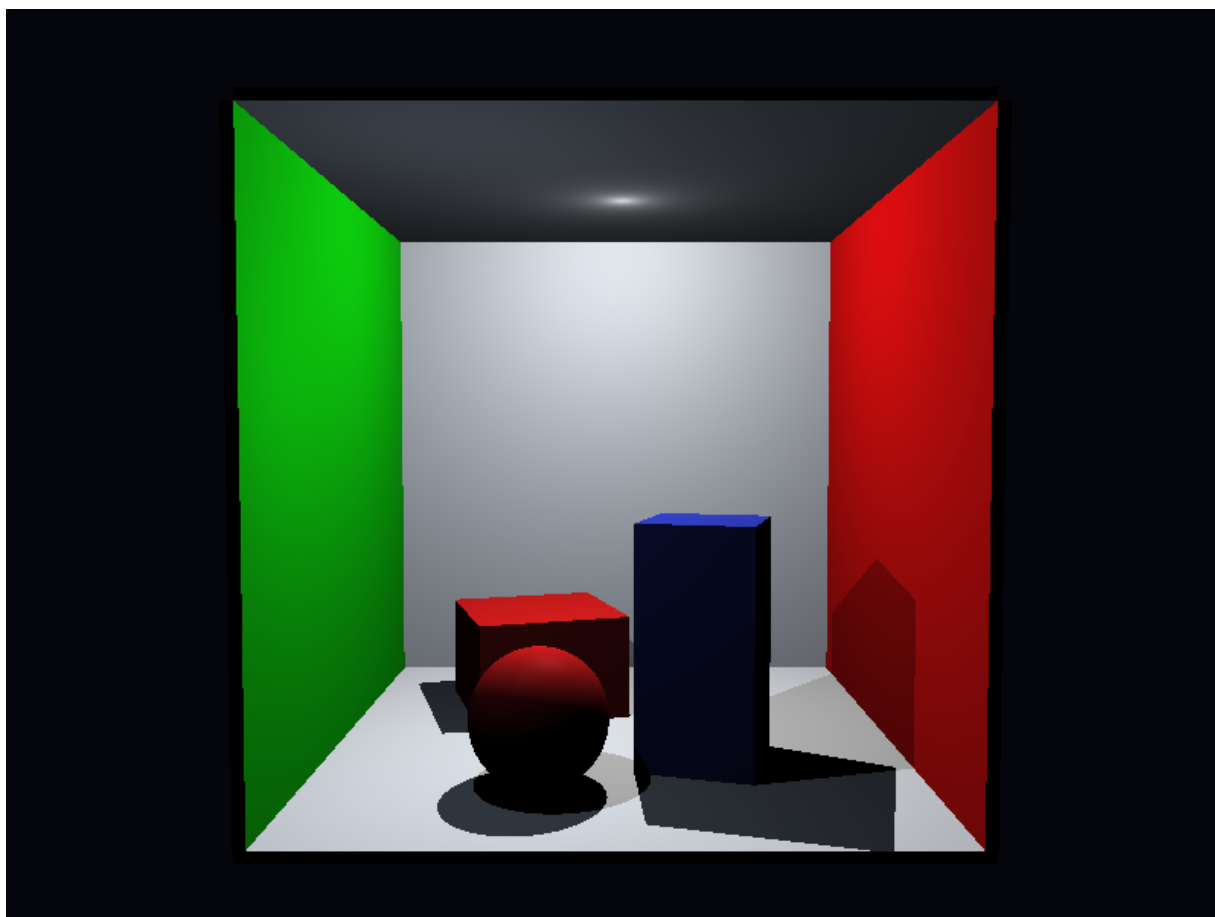


Figura 21. Versão ampliada da Figura 4: Phong com sombras diretas.

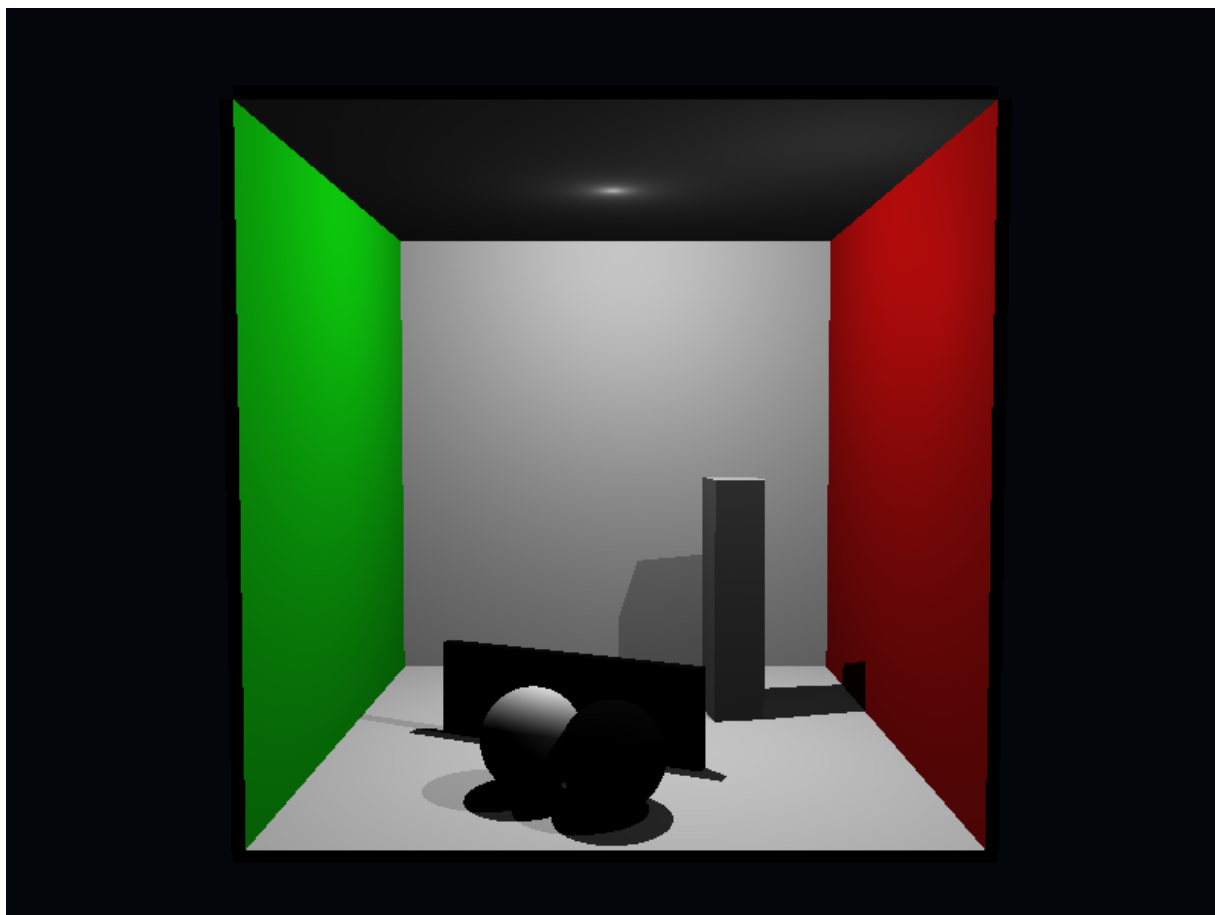


Figura 22. Versão ampliada da Figura 5: baseline center para comparação de antialiasing.

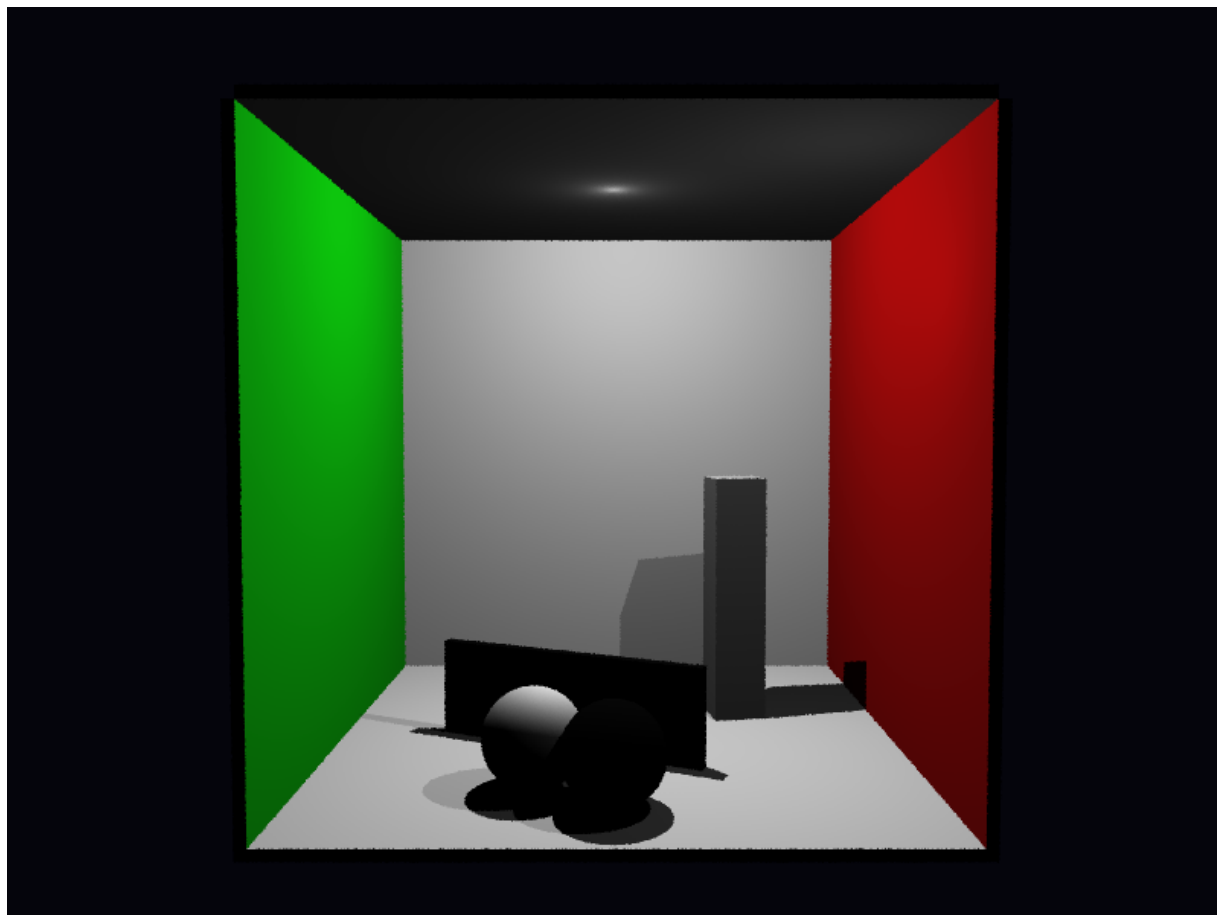


Figura 23. Versão ampliada da Figura 6: cena de antialiasing com jittered.

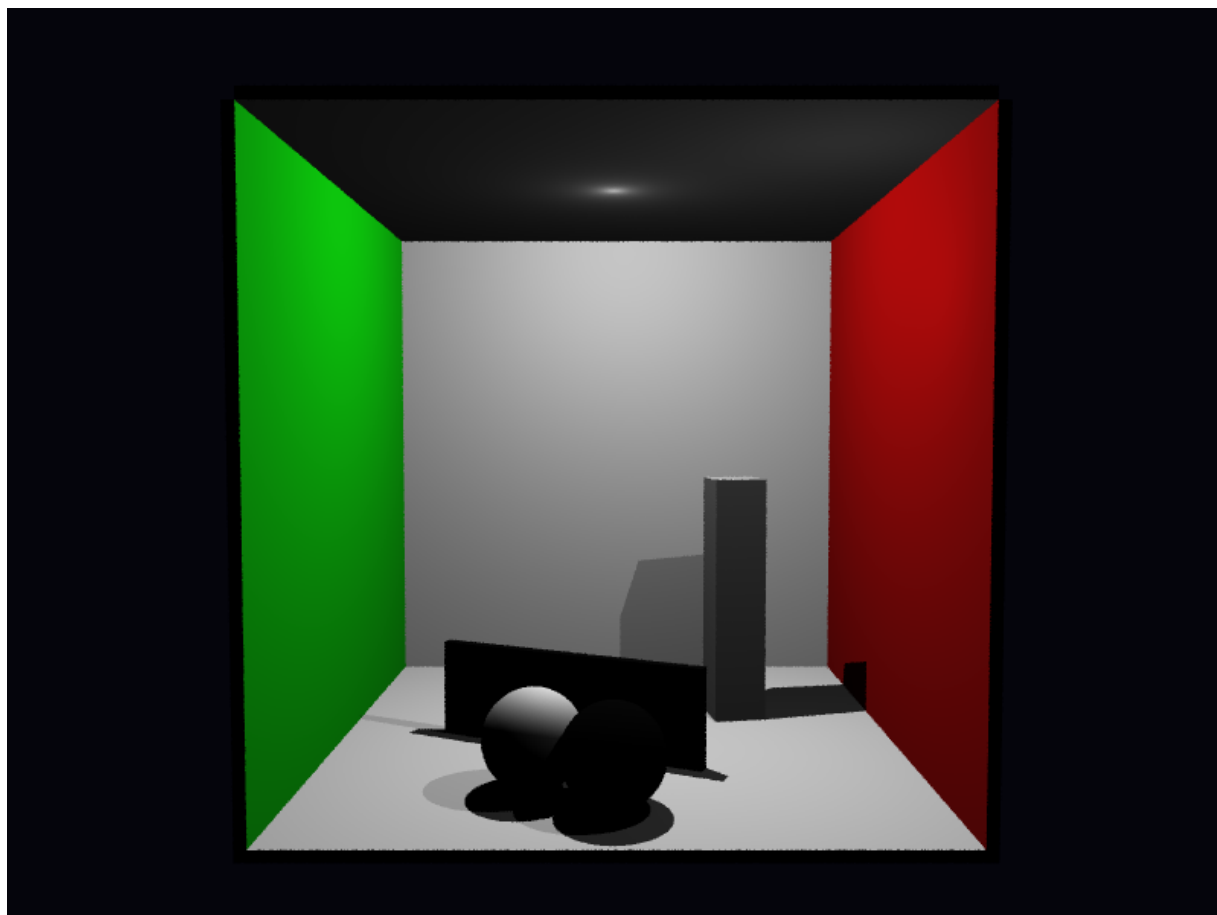


Figura 24. Versão ampliada da Figura 7: cena de antialiasing com stratified.

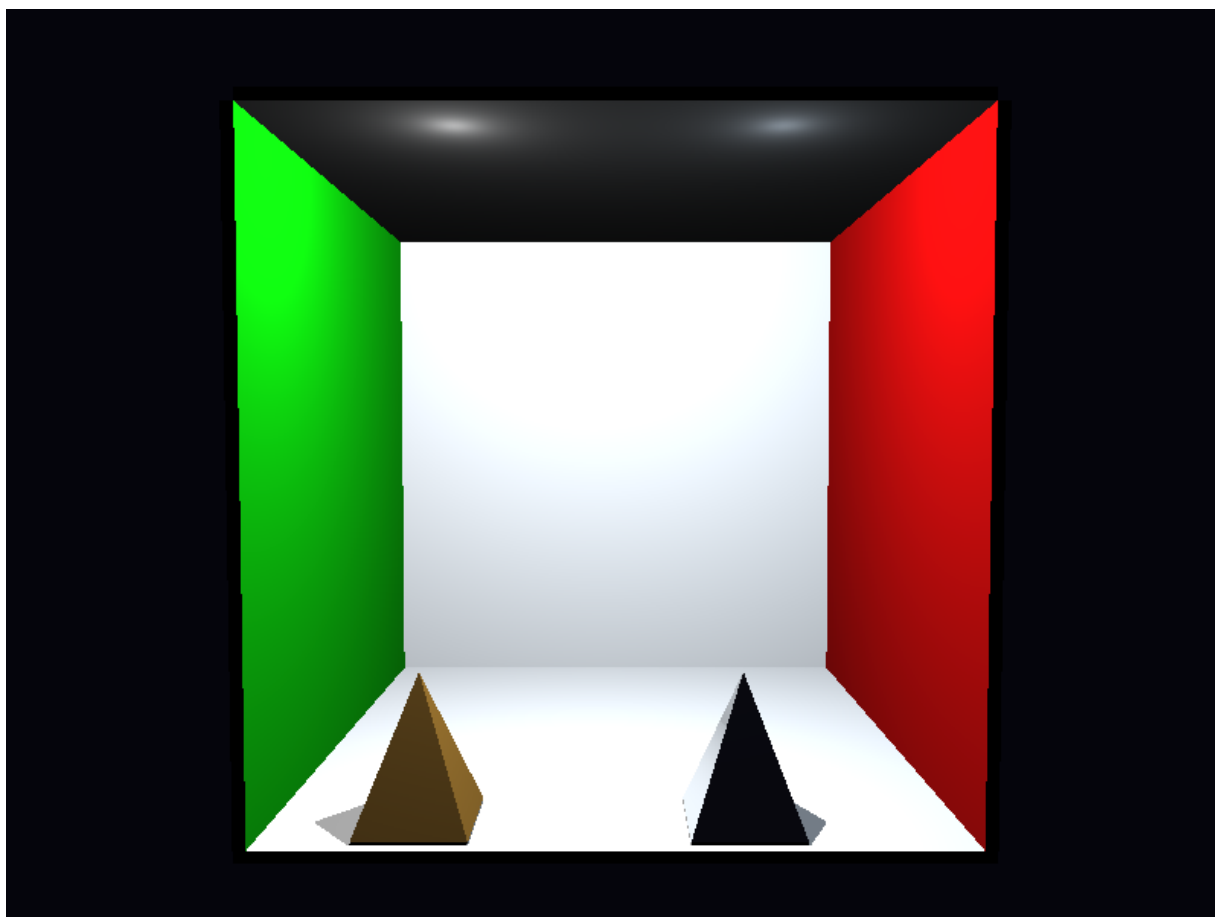


Figura 25. Versão ampliada da Figura 8: malha triangular com BVH local.

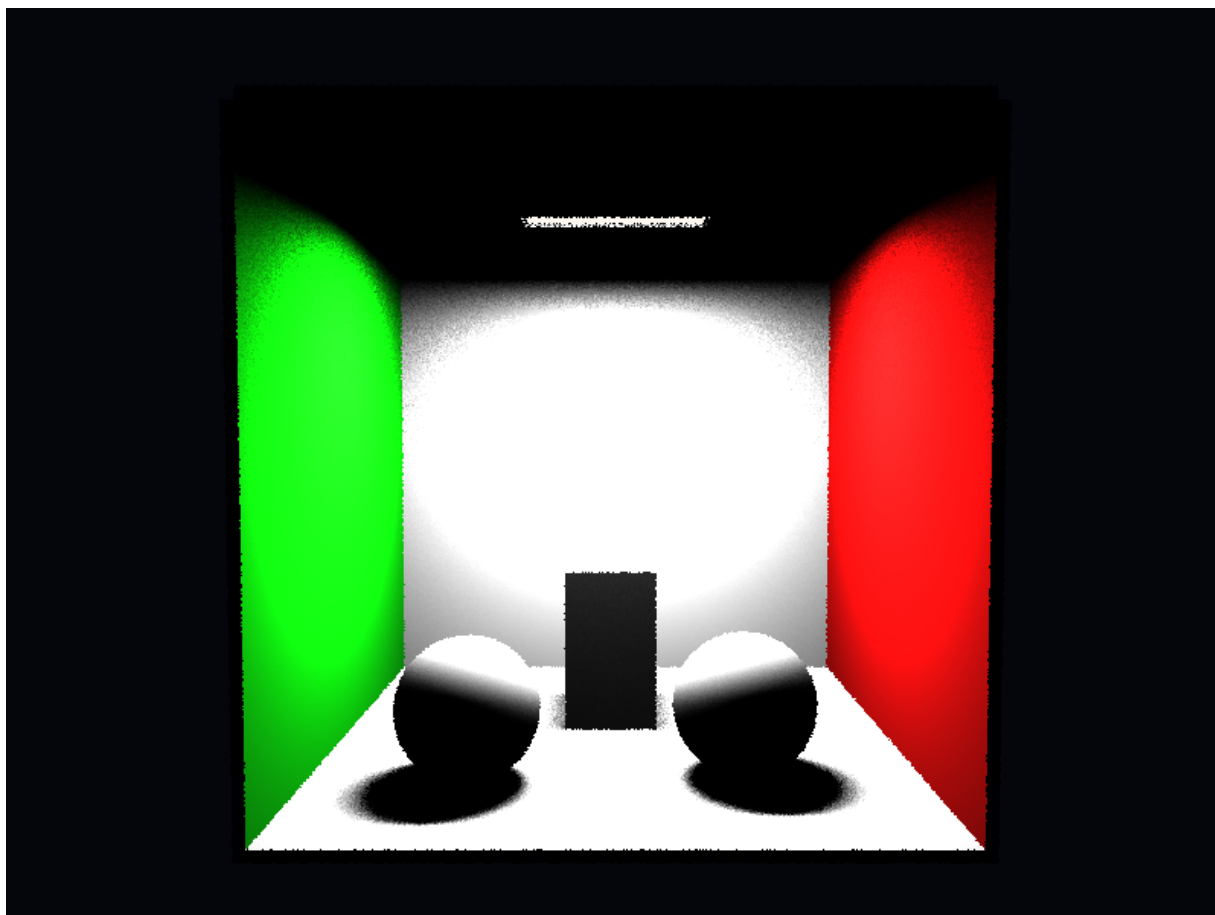


Figura 26. Versão ampliada da Figura 9: luz de área e emissor visível.

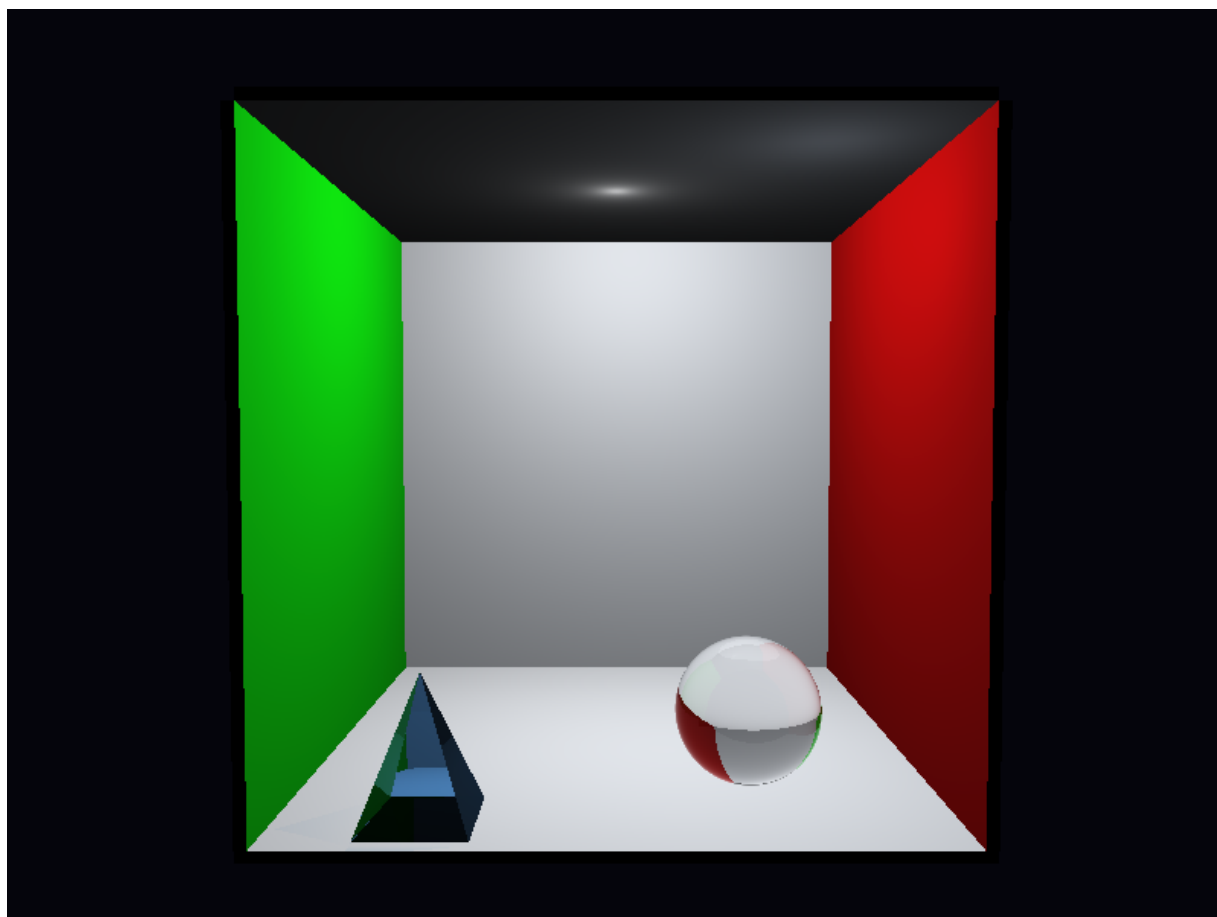


Figura 27. Versão ampliada da Figura 10: refração, TIR e Beer-Lambert.

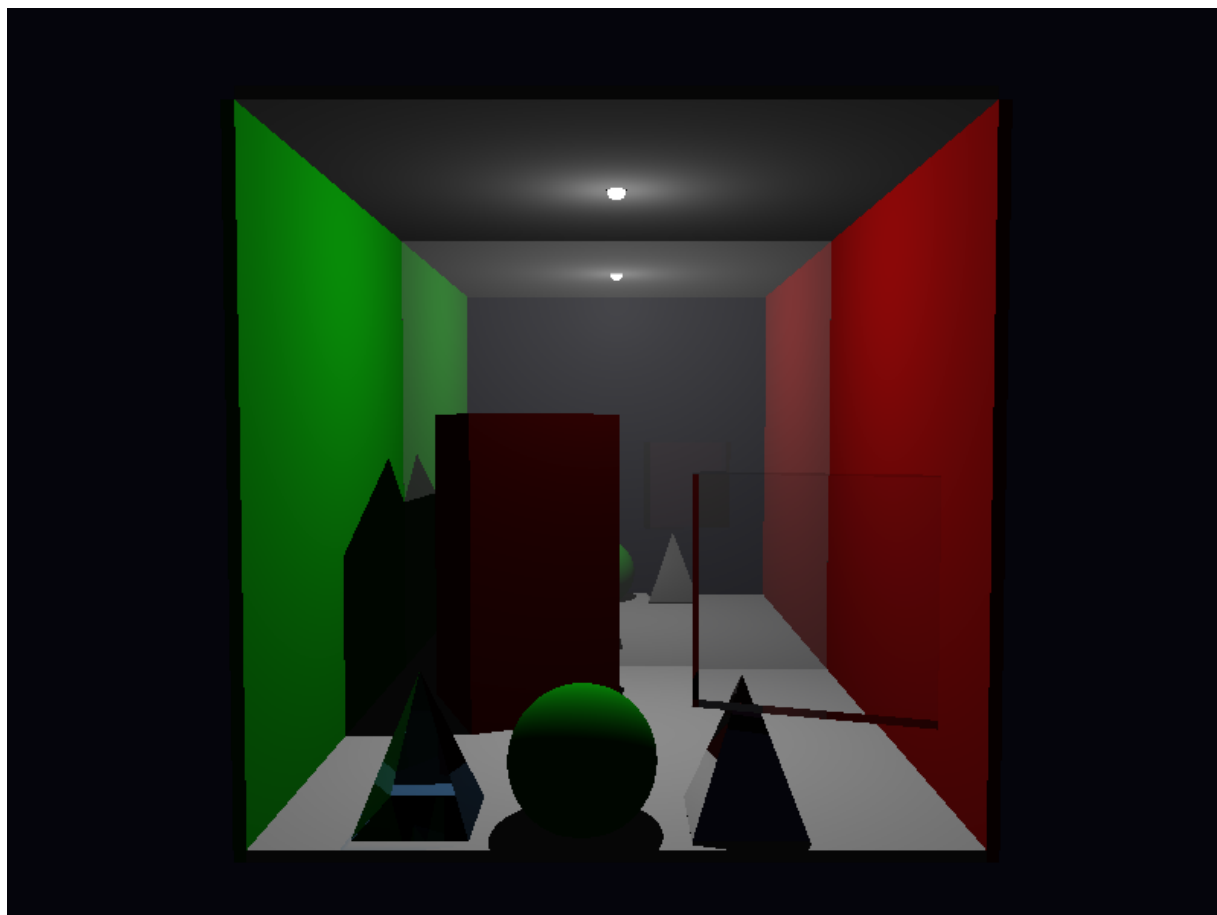


Figura 28. Versão ampliada da Figura 11: cena integrada no baseline center.

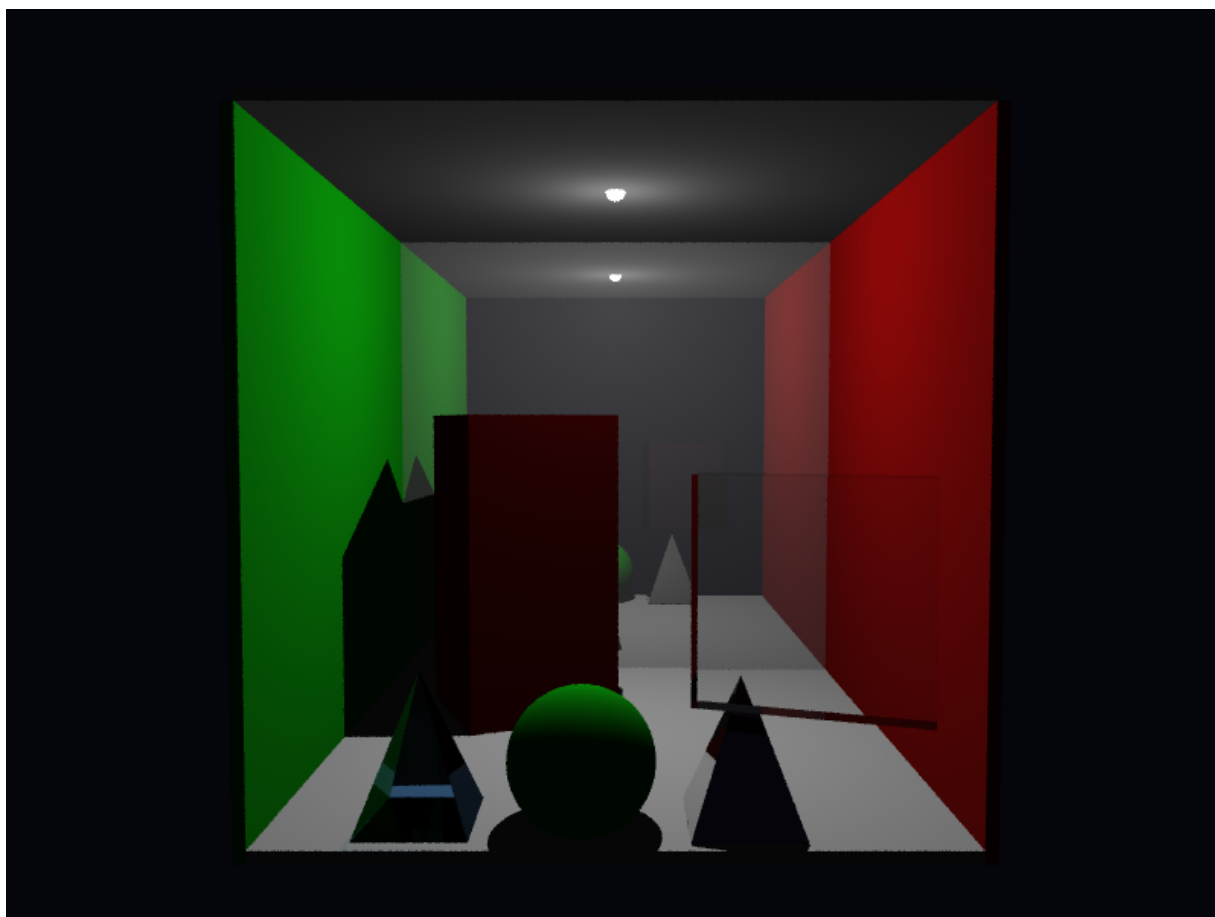


Figura 29. Versão ampliada da Figura 12: cena integrada com stratified.

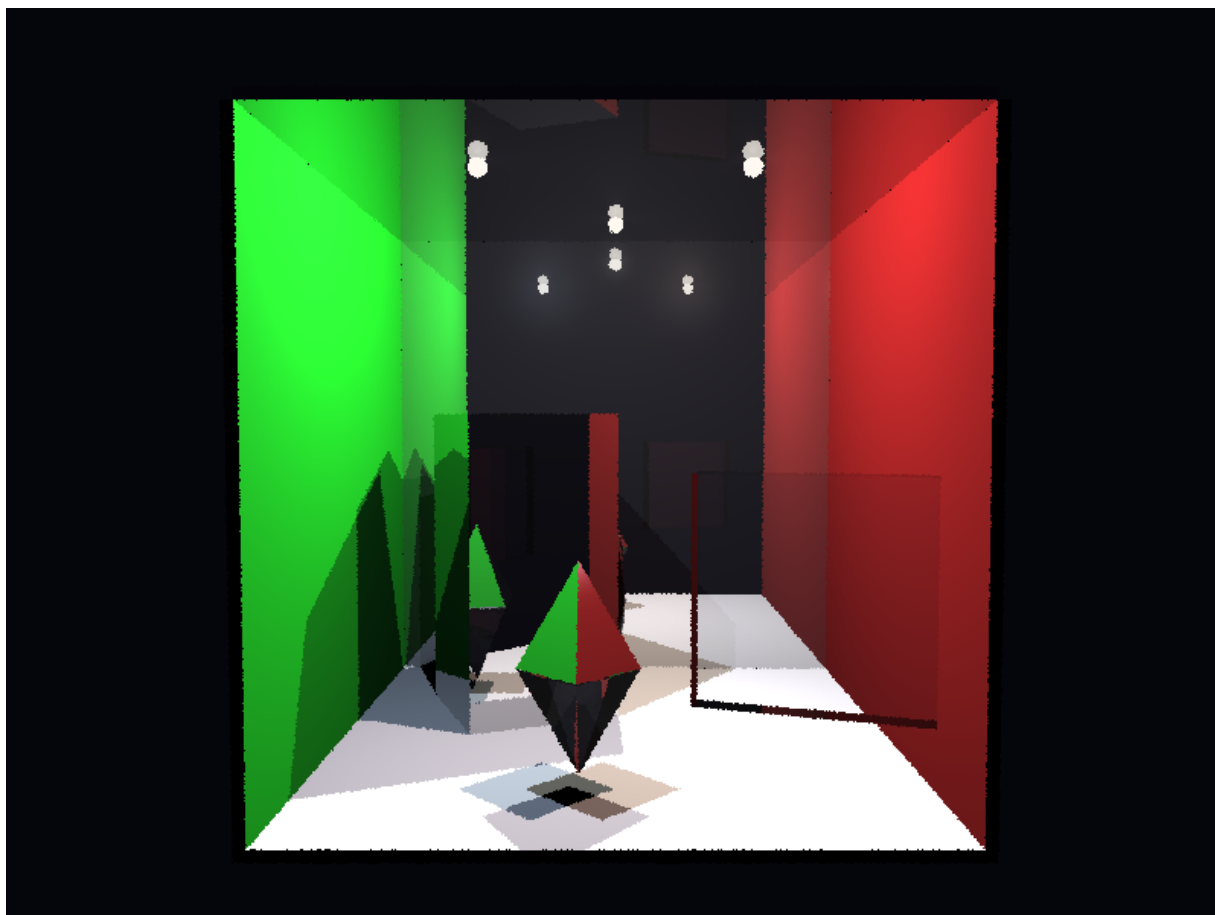


Figura 30. Versão ampliada da Figura 13: composição final integrada proj1_final.

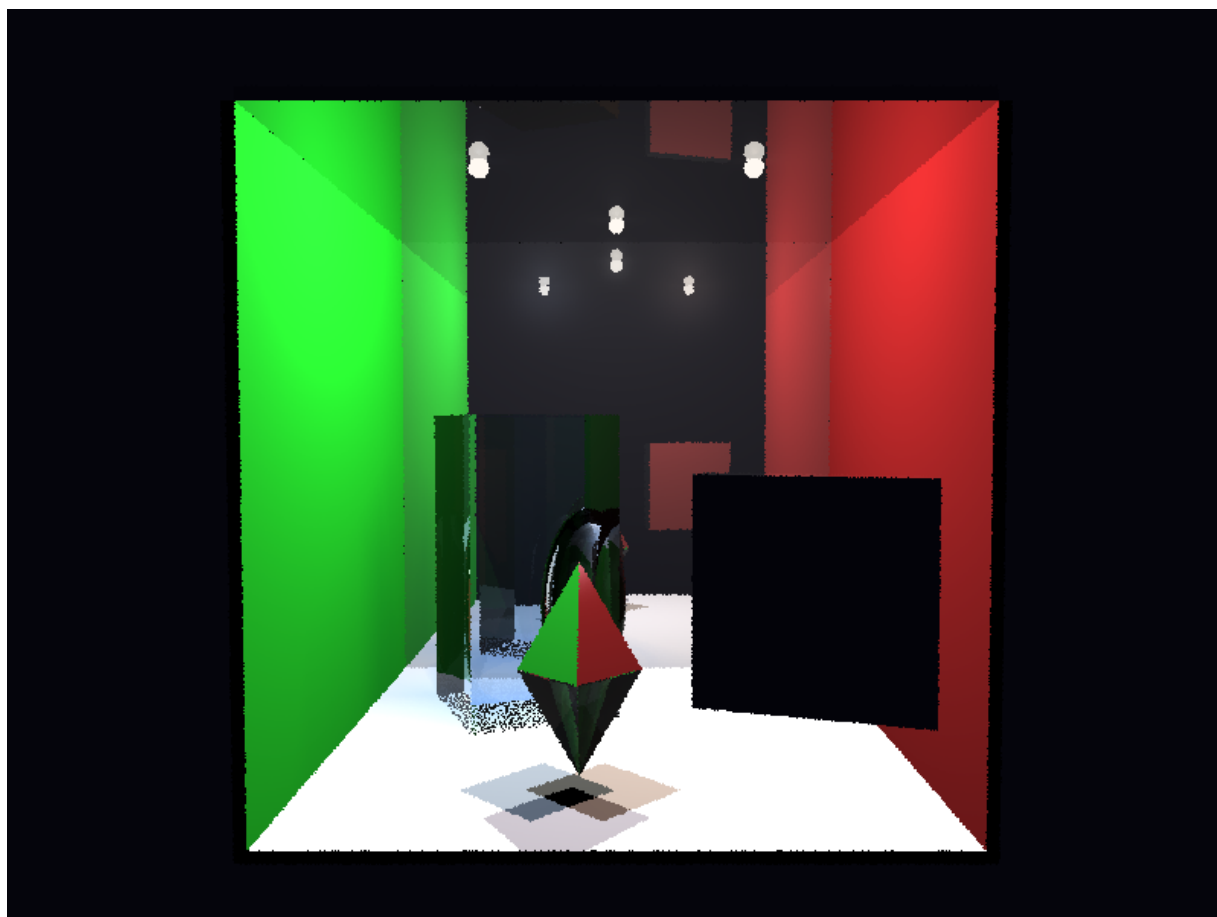


Figura 31. Versão ampliada da Figura 14: variante integrada proj1_final_v2.

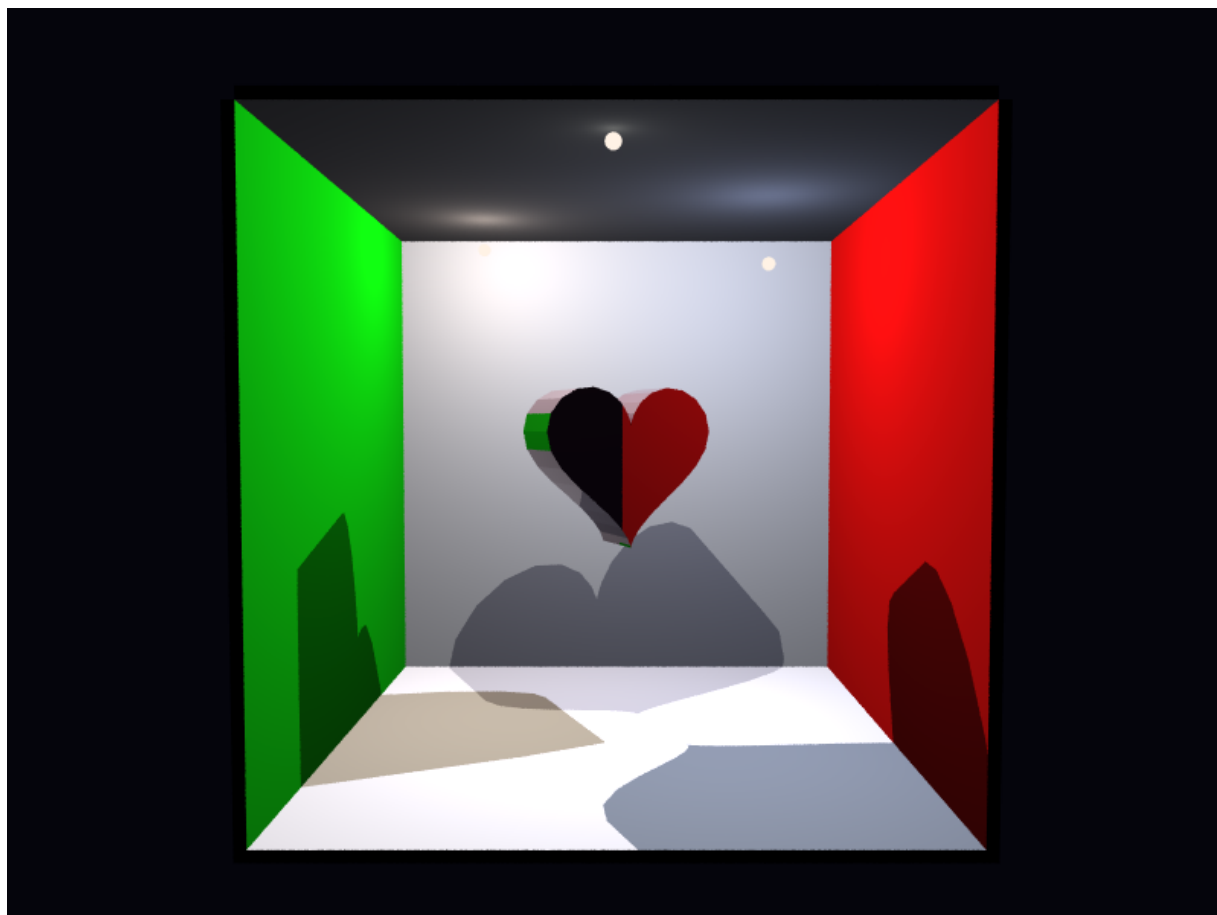


Figura 32. Versão ampliada da Figura 15: malha cardíaca triangulada com bvh.

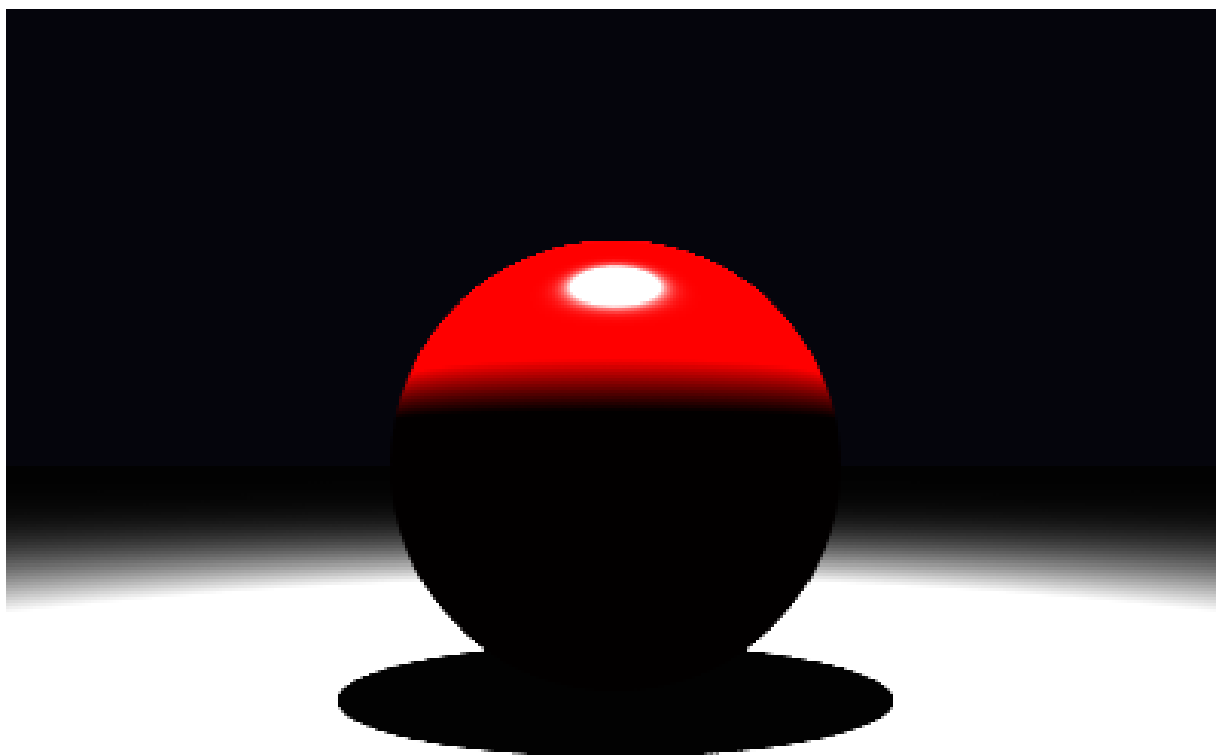


Figura 33. Artefato inicial de `main_scene`: cena mínima com esfera sobre plano e `properties.md` em formato explicativo.

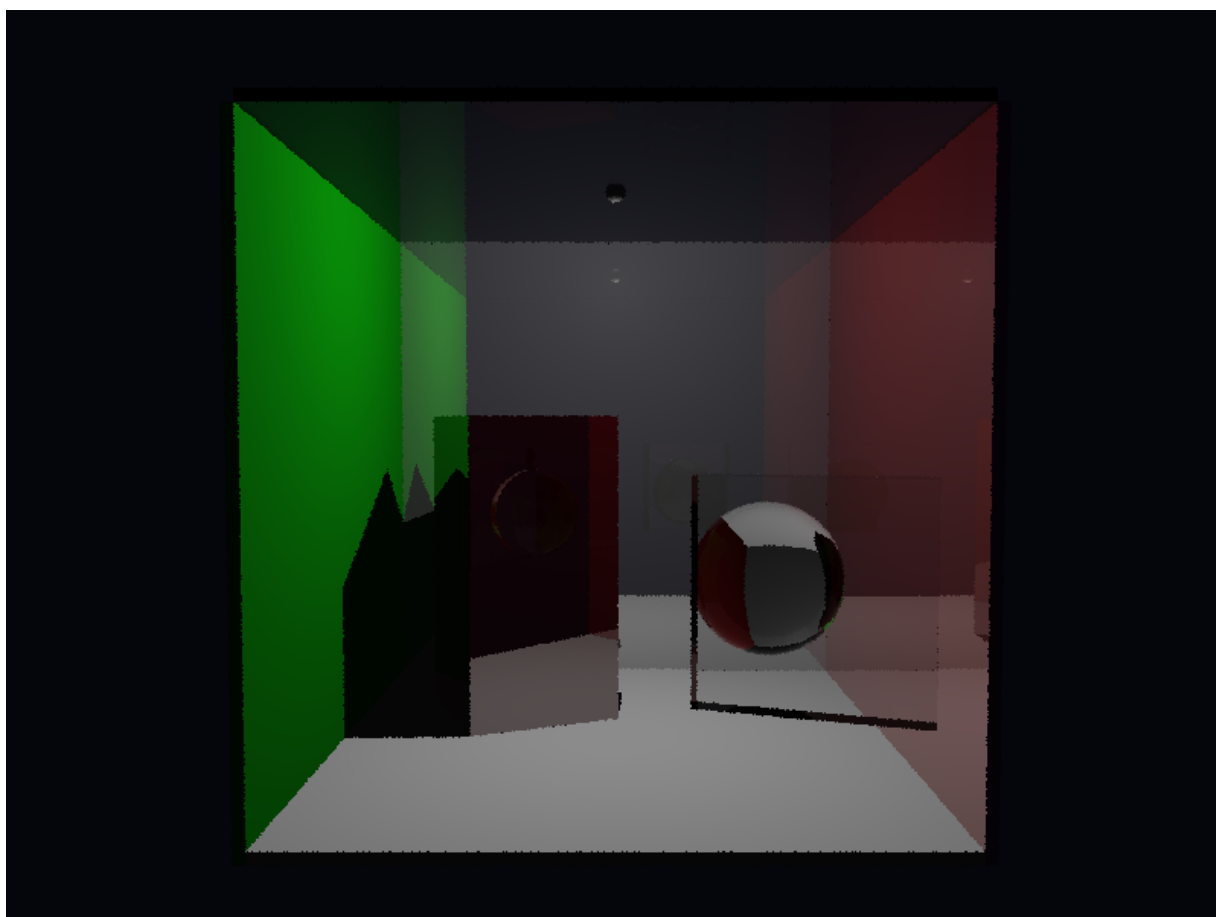


Figura 34. Variação exploratória de `cornell_box`: esfera transparente frontal e composição interna preservadas por snapshot.

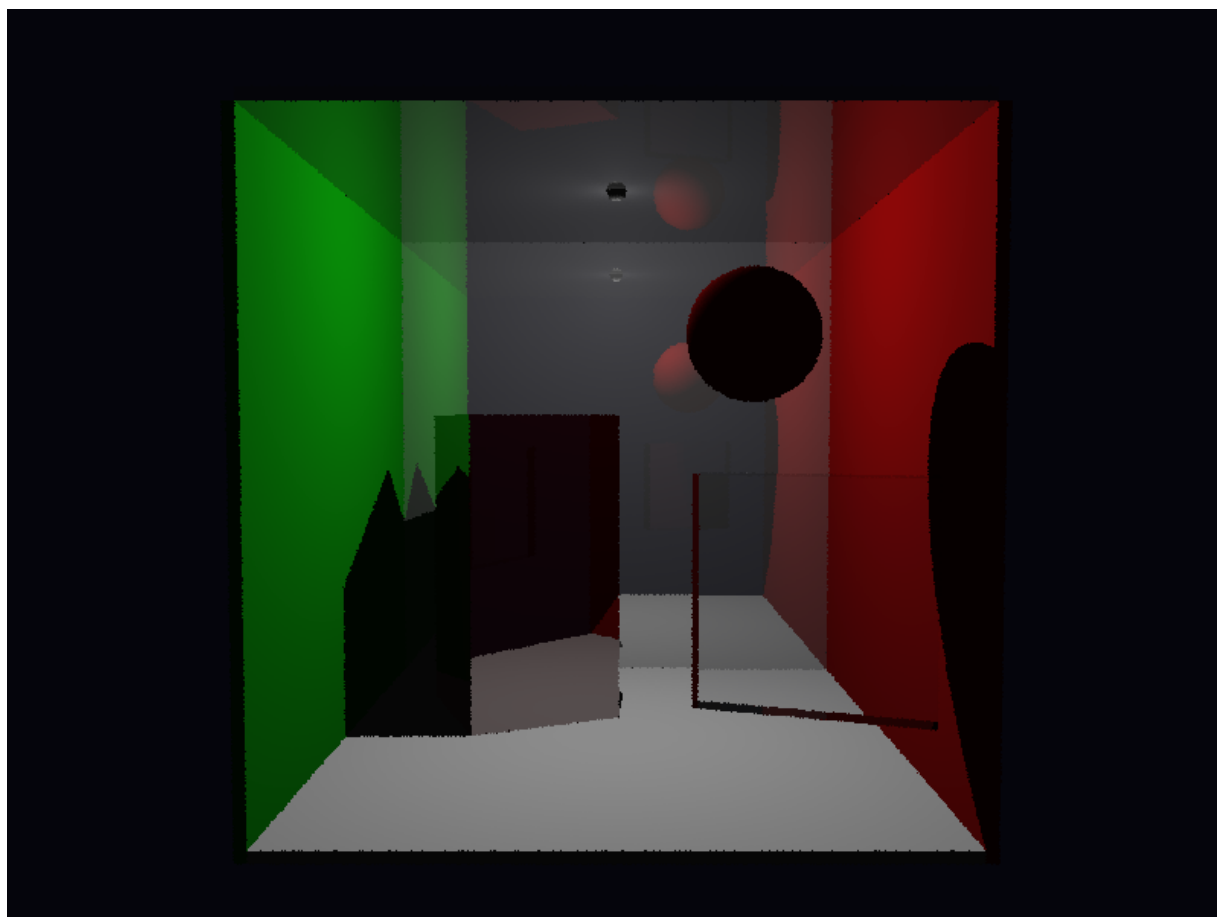


Figura 35. Outra variação de `cornell_box`: esfera refletiva em destaque e plano transparente interno.

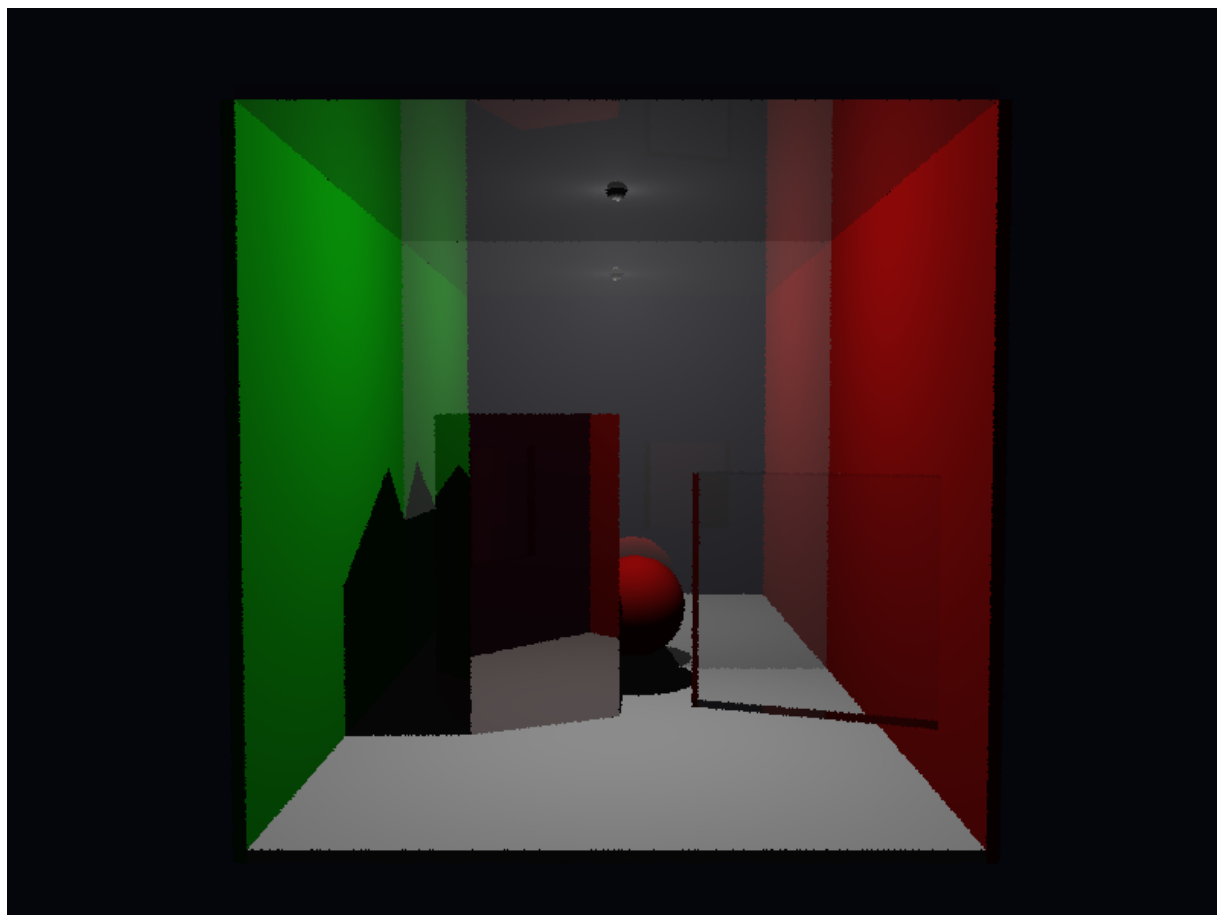


Figura 36. Variação de `cornell_box` com esfera vermelha deslocada para a região frontal da cena.

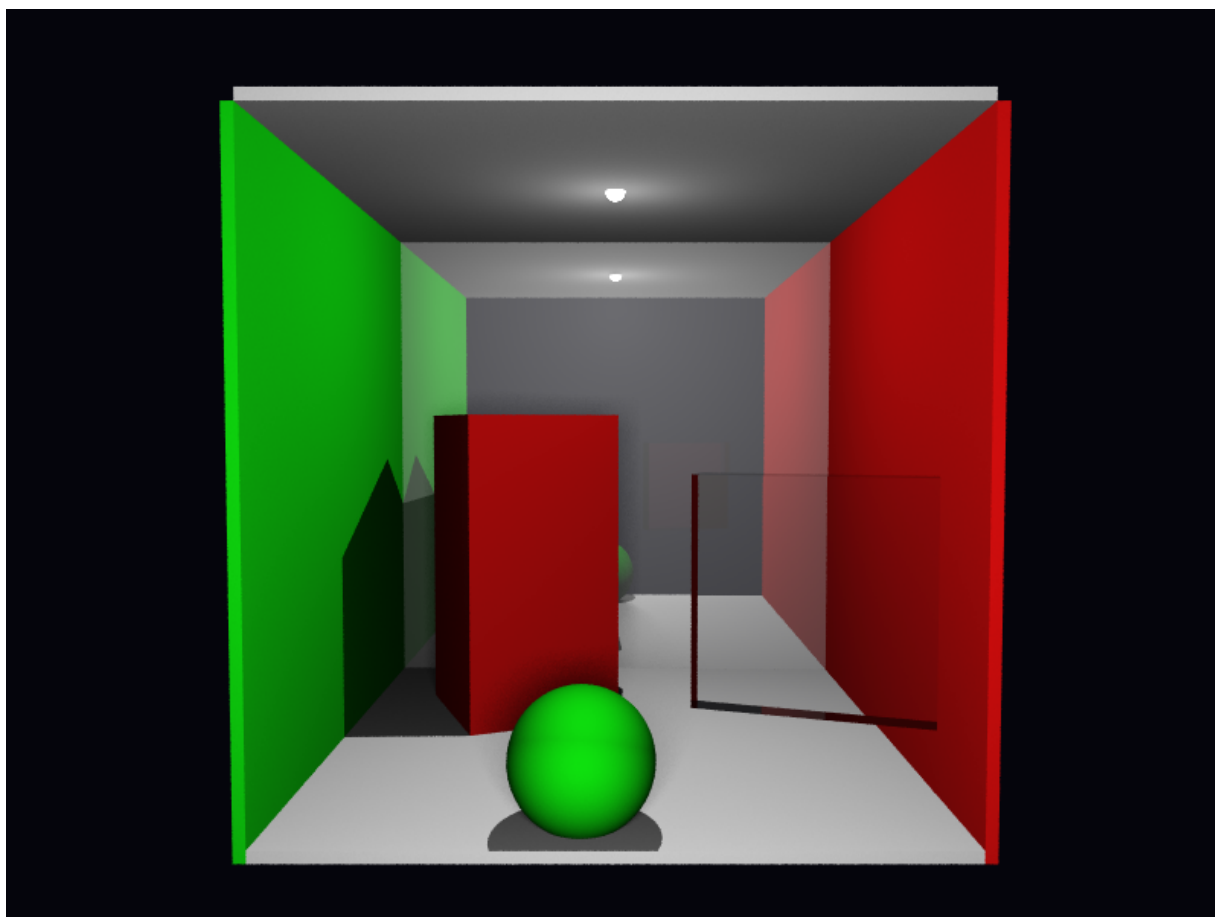


Figura 37. Variação de `cornell_box` com 25 amostras em `jittered` e múltiplas luzes pontuais.

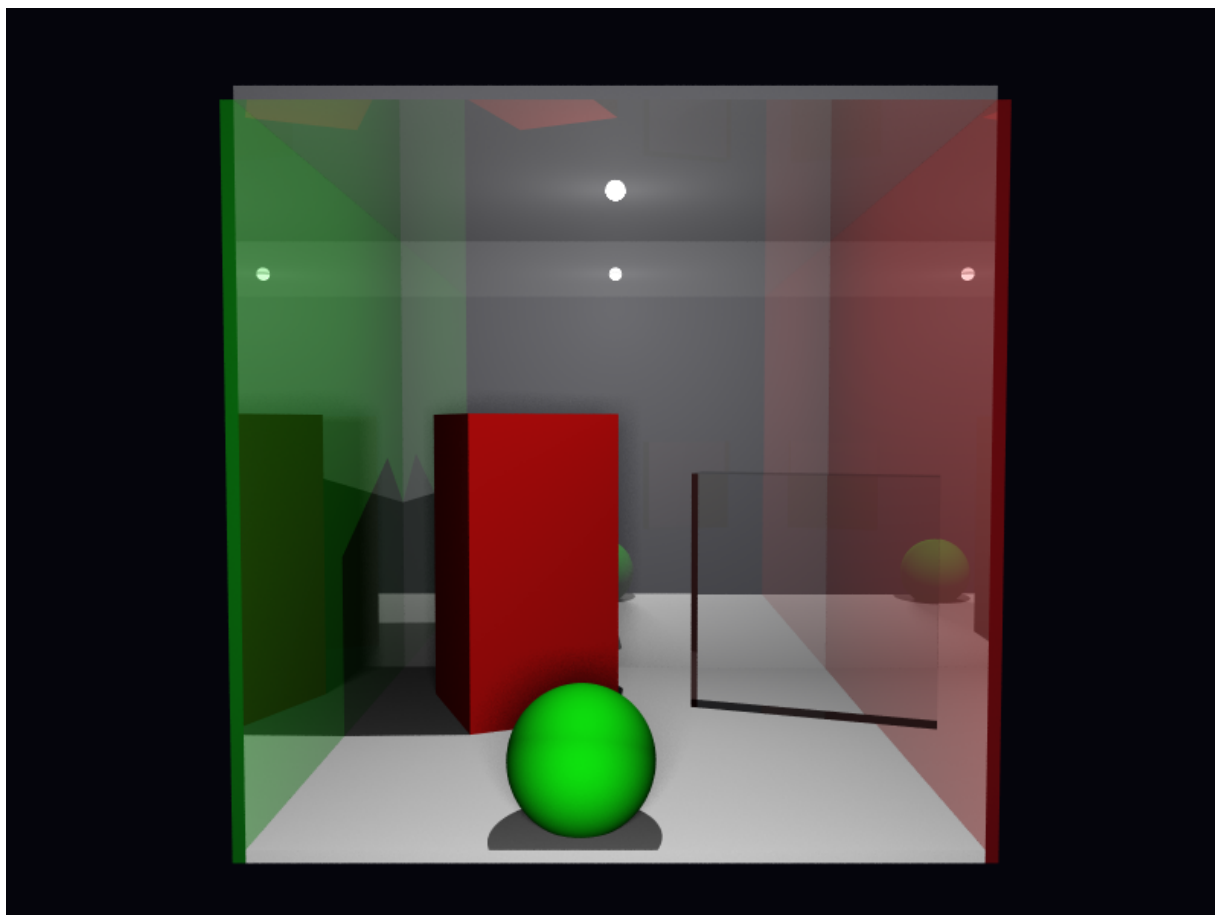


Figura 38. Variação de `cornell_box` com 25 amostras em `stratified` e paredes refletivas.

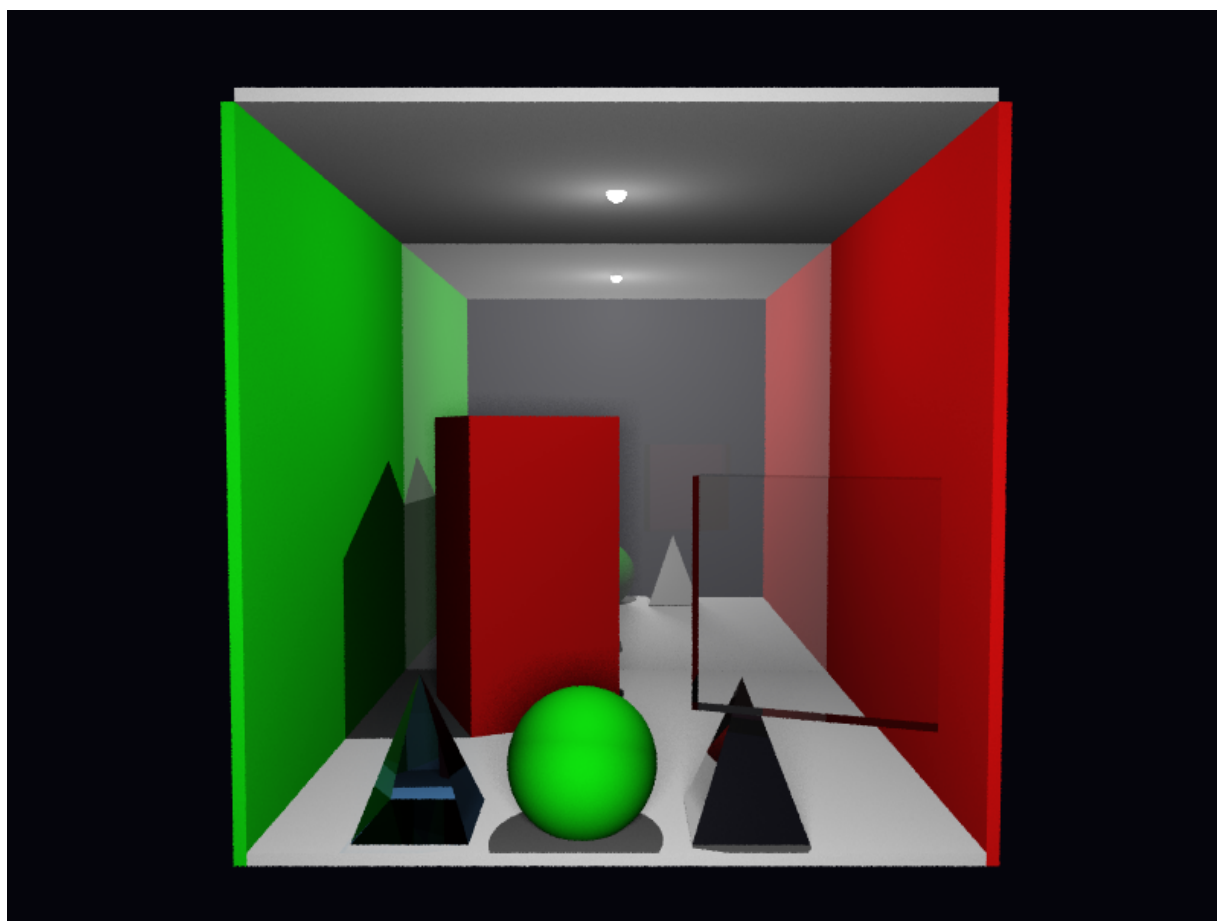


Figura 39. Variação adicional de `cornell_box_pyramid`: 10 amostras em `jittered`, malhas triangulares e luz de área estratificada.